

بخش دوم: مدیریت کوله پشتی ماجراجو

عالی! ماجراجویی ما تا اینجا فوق العاده بوده است.

تو در فصل قبل، به پایتون قدرت تصمیم‌گیری دادی و اولین بازی خودت را ساختی. حالا با `if`، `while`، `input` و کیمیاگری کاملاً آشنایی.

اما تابه حال، جعبه‌های جادویی (متغیرهای) ما فقط می‌توانستند یک غنیمت را در خود نگه دارند، مثل `coin = 14`.

اگر در یک ماجراجویی، ناگهان یک صندوقچه پراز آیتم‌های مختلف پیدا کنی چه؟ یک شمشیر، یک سپر، ده معجون، یک نقشه و... آیا می‌خواهی برای هر کدام یک متغیر جدا بسازی؟ `item1 = "sword"`، `item2 = "shield"` و...؟ این که خیلی نامرتبه!

ما به یک کوله‌پشتی جادویی نیاز داریم. یک کوله‌پشتی بزرگ که بتواند همهٔ غنائم دیگر را به صورت مرتب در خود نگه دارد.

وقت آن است که ماجراجویی جدیدی شروع کنیم!

فصل ۵: کوله‌پشتی جادویی: مدیریت غنائم ماجراجو

مأموریت فصل: ساخت یک برنامه برای مدیریت لیست خرید.

نقشه راه:

- کوله‌پشتی جادویی (Lists): قدرتمندترین ابزار برای نگهداری آیتم‌ها؛
- شروع ماجراجویی با کوله‌پشتی پُر (Initialized List)؛
- پر کردن کوله‌پشتی: اضافه کردن، دیدن و حذف کردن آیتم‌ها.
- بازرسی کوله‌پشتی: طلسم‌های `len()` و `in`؛
- سرشماری غنائم (حلقه `for`): ابزاری برای خواندن و مدیریت کوله‌پشتی؛
- پروژه: ساخت برنامه مدیریت لیست خرید با قابلیت افزودن و حذف آیتم.

۱. کوله‌پشتی جادویی (Lists)

تابه حال ما با داده‌های پایه، مثل int (اعداد) و string (رشته‌ها) کار می‌کردیم. حالا با اولین داده غیرپایه^۱ آشنا می‌شویم: لیست^۲.

لیست، دقیقاً مثل یک کوله‌پشتی است:

- یک لیست مرتب از آیتم‌هاست (ترتیب آیتم‌ها مهم و تکرار آنها هم مجاز است).
- می‌تواند شامل انواع مختلف داده باشد (عدد، متن یا حتی لیست‌های دیگر!).

ساختن یک کوله‌پشتی (لیست) جدید، با استفاده از براکت^۴ [] انجام می‌شود:

```
1. # Our backpack is empty at the start of the adventure
2. # We can create an empty list like this:
3. backpack = []
4. print(backpack)
```

خروجی:

```
[]
```

۲. شروع ماجراجویی با کوله‌پشتی پر (Initialized List)

گاهی اوقات، ما نمی‌خواهیم با دست خالی سفرمان را شروع کنیم! شاید از قبل مقداری غذا، نقشه یا سکه داشته باشیم. برای ساختن لیستی که از همان ابتدا دارای مقدار است، کافیست آیتم‌ها را درون براکت قرار دهیم و آنها را با علامت کاما^۵ (,) از هم جدا کنیم.

به یاد داشته باشید که در پایتون، انواع مختلف داده (مثل متن، عدد، لیست و...) می‌توانند دوستانه کنار هم در یک کوله‌پشتی بنشینند:

```
1. # Starting the adventure with a Map, a Flashlight, and 10 Gold Coins
2. backpack = ["Map", "Flashlight", 10]
3.
4. print(backpack)
```

¹ Primitive Types

² Non-Primitive Type

³ List

⁴ Bracket

⁵ Comma

خروجی:

```
['Map', 'Flashlight', 10]
```

نکته مهم: فراموش نکنید که بین هر آیتم یک کاما بگذارید تا پایتون بفهمد کجای کوله‌پشتی یک آیتم تمام شده و بعدی از کجا شروع می‌شود و اگر نگذارید با خطا روبه‌رو می‌شوید.

۳. پرکردن کوله‌پشتی (اضافه کردن، دیدن و حذف کردن)

یک کوله‌پشتی خالی به دردی نمی‌خورد. ما باید طلسم‌های جادویی مدیریت آن را یاد بگیریم.

۳-۱. طلسم `append()`: اضافه کردن آیتم

برای اضافه کردن یک آیتم به انتهای کوله‌پشتی، از طلسم `append()` استفاده می‌کنیم:

```
1. # Let's create our shopping list for the journey
2. shopping_list = []
3. print("List at start:")
4. print(shopping_list)
5.
6. # Now, let's add items using the .append() spell
7. shopping_list.append("sword") #
8. shopping_list.append("health_potion")
9. shopping_list.append("bread")
10.
11. print("List after adding items:")
12. print(shopping_list)
```

خروجی:

```
List at start:
[]
List after adding items:
['sword', 'health_potion', 'bread']
```

نکته: برای این طلسم خیلی مهم است که اول اسم لیست، بعد نقطه و سپس اسم طلسم بیاید (در آینده و در فصل شیء‌گرایی با این موضوع که چرا بعضی طلسم‌ها این‌طوری نوشته می‌شوند بیشتر آشنا می‌شویم).

۳-۲. خواندن محتوای آیتم‌ها با ایندکس^۶

چطور آیتم اول را ببینیم؟ برای این کار از ایندکس (شماره موقعیت) استفاده می‌کنیم.

Commented [GG11]: بهتر نیست جایی اشاره بشه که این طلسم ها در واقع به جور «متد» هستن؟

^۶Index

هشدار ماجراجو! در دنیای جادویی پایتون، شمارش از 0 (صفر) شروع می‌شود، نه از 1. اولین آیتم در ایندکس [0] است. دومین آیتم [1] و....

```
1. shopping_list = ["sword", "health_potion", "bread"]
2.
3. # Access items by their index number
4. first_item = shopping_list[0] #
5. second_item = shopping_list[1]
6.
7. print("First item needed:")
8. print(first_item) # Output: sword
9. print("Second item needed:")
10. print(second_item) # Output: health_potion
11.
12. # What about the LAST item?
13. # You can use -1 to get the last item, -2 for the second-to-last, etc.
14. last_item = shopping_list[-1] #
15. print("Last item:")
16. print(last_item) # Output: bread
```

Commented [GG12]: نباید حذف شود؟

Commented [GG13]: نباید حذف شود؟

۳-۳. طلسم `remove()`: حذف کردن آیتم

اگر یک آیتم را خریدیم و بخواهیم آن را از لیست خرید حذف کنیم چی؟ اگر نام آیتم را بدانیم، از طلسم `remove()` استفاده می‌کنیم.

```
1. shopping_list = ["sword", "health_potion", "bread"]
2. print("Original list:")
3. print(shopping_list)
4.
5. # We found a health potion! Let's remove it from the list
6. shopping_list.remove("health_potion")
7.
8. print("List after removing potion:")
9. print(shopping_list)
```

خروجی:

```
Original list:
['sword', 'health_potion', 'bread']
List after removing potion:
['sword', 'bread']
```

نکته مهم: اگر مقداری که می‌خواهیم آن را حذف کنیم وجود نداشته باشد، پایتون به ما ارور می‌دهد. (برای جلوگیری از این ارور، در ادامه یاد می‌گیریم که ابتدا با `in` بررسی کنیم که آیا مقدار موجود است و بعد در صورت وجود، آن را حذف می‌کنیم).

۴. بازرسی کوله‌پشتی (طلمس‌های len و in)

گاهی اوقات ما نمی‌خواهیم آیتمی را اضافه یا حذف کنیم، بلکه فقط به اطلاعات در مورد کوله‌پشتیمان نیاز داریم. مثلاً:

- کوله‌پشتی من چقدر سنگین شده؟ (چند آیتم داخل آن است؟)
- آیا من قبلاً شمشیر را برداشته‌ام؟ (آیا آیتم x در لیست وجود دارد؟)

برای این دو سؤال حیاتی، دو طلمس جادویی جدید داریم:

۴-۱. طلمس len(): طلمس شمارش‌گر

اگر بخواهی بدانی دقیقاً چند آیتم در لیست وجود دارد، می‌توانی از طلمس **len()** (مخفف Length، به معنی طول) استفاده کنی.

```
1. shopping_list = ["sword", "health_potion", "bread"]
2. empty_list = []
3.
4. # Get the number of items in the list
5. item_count = len(shopping_list)
6. empty_count = len(empty_list)
7.
8. print("Items in shopping list:")
9. print(item_count) # Output: 3
10.
11. print("Items in empty list:")
12. print(empty_count) # Output: 0
```

این طلمس بسیار کاربردی‌ست و در ادامه زیاد از آن استفاده می‌کنیم.

۴-۲. طلمس in: طلمس جست‌وجوگر

یکی از قدرتمندترین طلمس‌های پایتون است. اگر بخواهی چک کنی آیا یک آیتم خاص داخل (in) لیست هست یا نه، از این طلمس استفاده می‌کنی. نتیجه این طلمس یک بولین، یعنی True یا False است.

```
1. shopping_list = ["sword", "health_potion", "bread"]
2.
3. # Check if 'sword' is in the list
4. has_sword = "sword" in shopping_list
5. print("Do I have a sword?")
6. print(has_sword) # Output: True
7.
8. # Check if 'shield' is in the list
9. has_shield = "shield" in shopping_list
10. print("Do I have a shield?")
11. print(has_shield) # Output: False
```

نکته کاربردی: اگر ماجراجو بخواهد آیتمی را حذف کند که در لیست موجود نیست، پایتون ارور می‌دهد. پس قبل از حذف، اول با `if` وجود آن را بررسی می‌کنیم و فقط اگر `True` بود، آن را `remove` می‌کنیم.

```
1. if item_to_remove in shopping_list:
2.     shopping.remove(item_to_remove)
```

۵. سرشماری غنائم (حلقه for)

ما قبلاً از حلقه `while` برای تکرار کد استفاده کردیم. `while` عالی بود چون تا زمانی که یک شرط `True` باشد به تکرار ادامه می‌داد. اما برای لیست، اغلب می‌خواهیم یک کار ساده انجام دهیم، مثلاً: «به ازای هر آیتم در کوله‌پشتی، آن را به من نشان بده».

انجام این کار با `while` کمی سخت است. اما ما یک طلسم جادویی جدید و بسیار ساده‌تر برای این کار داریم: **حلقه for**.

حلقه `for` برای گشتن^۷ روی یک دنباله^۸ مثل لیست ساخته شده است.

```
1. shopping_list = ["sword", "health_potion", "bread"]
2.
3. print("--- Shopping List (one by one) ---")
4.
5. # The 'for' loop:
6. # "item" is a temporary variable we create.
7. # Python will put 'sword' in 'item', run the code.
8. # Then put 'health_potion' in 'item', run the code.
9. # Then put 'bread' in 'item', run the code.
10. for item in shopping_list:
11.     print("I need to buy a " + item)
12.
13. print("-----")
```

Commented [GG14]: نباید حذف شود؟

خروجی:

```
--- Shopping List (one by one) ---
I need to buy a sword
I need to buy a health_potion
I need to buy a bread
-----
```

کار این حلقه بسیار تمیزتر از `while` است!

^۷Iterate

^۸Sequence

۶. پروژه: ساخت سیستم مدیریت کوله‌پشتی^۹ – 100 xp

ماجراجو، تبریک می‌گوییم! تو آماده‌ای. در این مأموریت، قرار نیست ابزار جدیدی یاد بگیریم؛ در عوض، می‌خواهیم تمام قدرت‌هایی که در فصل‌های گذشته به دست آوردیم (مثل لیست‌ها، حلقه‌ها و شرط‌ها) را با هم ترکیب کنیم تا یک نرم‌افزار واقعی بسازیم.

این ابزار، یک دستیار هوشمند برای مدیریت تجهیزات و کوله‌پشتی ما در طول سفر خواهد بود.

۶-۱. تصویر نهایی: این برنامه قرار است چه کار کند؟

قبل از اینکه وارد کدهای پیچیده شویم، بیا یک لحظه چشمانمان را ببندیم و نتیجه نهایی را تصور کنیم. تا الان، برنامه‌هایمان اجرا می‌شدند، تنها یک کار انجام می‌دادند و سریعاً بسته می‌شدند. اما این بار یک برنامه تعاملی^{۱۰} و همیشه بیدار می‌سازیم.

تصور کن وسط یک جنگل تاریک ایستاده‌ای و این برنامه مثل یک روح نگهبان کنار توست و تا وقتی نگویی خداحافظ، منتظر دستورات می‌ماند:

- اگر بگویی **add** (بردار): آماده می‌شود تا اسم آیتم (مثلاً شمشیر یا معجون) را بگیرد و در کوله‌پشتی بگذارد.
- اگر بگویی **show** (نشان بده): لیست تمام چیزهایی که در کوله‌پشتی داری را خیلی مرتب نشان می‌دهد.
- اگر بگویی **remove** (دور بینداز): آیتمی که مصرف کردی یا نیاز نداری را از کوله‌پشتی بیرون می‌اندازد.
- نکته حیاتی: تا وقتی خودت نگویی **quit** (استراحت)، برنامه بسته نمی‌شود و دوباره می‌پرسد: «خب، دستور بعدی چیست؟».

۶-۲. نقشه و طرح اولیه

یک برنامه‌نویس حرفه‌ای اول فکر می‌کند، بعد کد می‌زند. بیا ببیند منطق برنامه را مهندسی کنیم. ما برای ساخت این سیستم به ۴ ستون اصلی نیاز داریم:

Commented [GG15]: بهتر است معادل انگلیسی این اصطلاح

هم در پاورقی بیاید. Always on persiatant مثلاً

^۹ Backpack Manager

^{۱۰} Interactive

۱. ساخت حافظه (مکان امن برای آیتم‌ها): اول از همه به جایی نیاز داریم که تجهیزات (تیرکمان، نقشه و...) را در آن ذخیره کنیم.

- **منطق:** یک لیست خالی [] می‌سازیم.

- نکته حیاتی (خطر فراموشی): این لیست را باید قبل از شروع حلقه بسازیم.

○ چرا؟ اگر لیست را داخل حلقه بسازی، هربار که برنامه می‌چرخد و سؤال می‌پرسد، یک کوله‌پشتی نو و خالی ساخته می‌شود و تمام آیتم‌های قبلی پاک می‌شوند! پس حافظه باید بیرون از چرخه باشد.

۲. روشن کردن موتور (حلقه تکرار): برنامه نباید بعد از یک دستور بخوابد! باید بیدار بماند.

- **منطق:** یک متغیر به نام `is_running` (آیا در حال اجراست؟) را روی `True` تنظیم می‌کنیم و یک حلقه `while` می‌سازیم که تا وقتی این متغیر روشن است، بچرخد.

۳. گوش دادن به فرمان (Input): داخل حلقه، ربات باید مؤدبانه بپرسد چه می‌خواهی.

- **منطق:** یک دستور `input` می‌نویسیم تا فرمان اصلی (`show.add`, ...) را بگیرد.

۴. مغز متفکر (تصمیم‌گیری): حالا باید فرمان را بررسی کنیم (اینجاست که `if` و `elif` وارد می‌شوند):

- اگر `add` بود: بپرس چی؟ ➡ اضافه کن.
- اگر `show` بود: با حلقه `for` چاپ کن.
- اگر `remove` بود: بپرس چی؟ ➡ حذف کن.
- اگر `quit` بود: کلید موتور (`is_running`) را خاموش کن.

۳-۶. چالش‌های فکری

این پروژه کمی سخت است، پس بیا قبل از کدنویسی، گره‌های ذهنی را باز کنیم:

- **چالش ۱. برداشتن واقعی:** وقتی کاربر دستور `add` را می‌دهد، کار تمام نشده‌است! برنامه تازه باید بپرسد: «چه آیتمی؟». پس به یک `input` دوم نیاز داریم. بعد از گرفتن اسم آیتم، با کدام دستور آن را به انتهای لیست می‌چسبانیم؟ (راهنمایی: با دستور `append()`).

Commented [GG16]: این منطق کاری است که انجام شده یا راهکار انجام آن؟

- چالش ۲. نمایش زیبا (نه زشت!): اگر کاربر گفت `show`، ما نمی‌خواهیم لیست را مدل کامپیوتری و زشت چاپ کنیم (مثل `['Sword', 'Potion']`)، می‌خواهیم اقلام زیرهم و مرتب چاپ گردد تا راحت خوانده شوند. کدام حلقه بود که روی تک‌تک آیتم‌های لیست قدم می‌زد؟ (راهنمایی: *حلقه `for item in list`*).
- چالش ۳. تله حذف کردن (باگ احتمالی): فرض کن کاربر می‌گوید: «سپر را حذف کن»، اما اصلاً سپری در کوله‌پشتی نیست! اگر همین‌طوری دستور حذف بدهیم، برنامه با صورت زمین می‌خورد! (ارور می‌دهد و بسته می‌شود). چطور می‌توانیم قبل از حذف، چک کنیم که آیا این آیتم اصلاً وجود دارد؟ (راهنمایی: *استفاده از کلمه کلیدی `in` داخل یک `if`*).
- چالش ۴. پایان بازی: چطور حلقه بی‌نهایت را متوقف کنیم؟ کافیست متغیر `is_running` را که مثل کلید برق عمل می‌کند، به `False` تغییر دهیم.

۶-۴. کد نهایی پروژه (The Final Spell)

وقتش رسیده که کلمات را به عمل تبدیل کنیم. کد زیر را با دقت بخوان و در فایل `project.py` بنویس. سعی کن به کامنت‌های انگلیسی دقت کنی تا بفهمی هر خط دقیقاً چه کار می‌کند.

```

1. # --- The Adventurer's Backpack Manager ---
2.
3. # 1. Setup the Memory (The Backpack)
4. # WARNING: We create the list OUTSIDE the loop.
5. # If we put it inside, it would be wiped clean every time the loop runs!
6. backpack = []
7. is_running = True
8.
9. print("Welcome to your Adventure Backpack!")
10. print("Commands: 'add', 'remove', 'show', 'quit'")
11.
12. # 2. Start the Engine (The Loop)
13. # The program stays alive as long as is_running is True
14. while is_running == True:
15.     print("-----")
16.
17.     # 3. Get command from user (Listening)
18.     command = input("What is your command? ")
19.
20.     # 4. Make decisions based on command (The Brain)
21.
22.     # --- ADD COMMAND (Pick up item) ---
23.     if command == "add":
24.         item = input("What item did you find? ")
25.         backpack.append(item)
26.         print(item + " added to backpack.")
27.
28.     # --- REMOVE COMMAND (Drop item) ---
29.     elif command == "remove":

```

```

30.         item = input("What item to drop? ")
31.
32.         # Safety Check: Is the item actually in the backpack?
33.         if item in backpack:
34.             backpack.remove(item)
35.             print(item + " dropped.")
36.         else:
37.             # If item is NOT in the list, tell the user instead of
38.             crashing
39.             print("Error: You don't have " + item + " in your backpack!")
40.
41.         # --- SHOW COMMAND (Check Inventory) ---
42.         elif command == "show":
43.             print("--- Your Inventory ---")
44.
45.             # Check if backpack is empty first
46.             if len(backpack) == 0:
47.                 print("Your backpack is empty.")
48.             else:
49.                 # Loop through the list to show items nicely
50.                 for item in backpack:
51.                     print("- " + item)
52.
53.         # --- QUIT COMMAND (Rest) ---
54.         elif command == "quit":
55.             print("Safe travels, Adventurer!")
56.             # This turns off the 'while' loop and stops the program
57.             is_running = False
58.
59.         # --- INVALID COMMAND ---
60.         else:
61.             # If the user types something like "dance" or "fly"
62.             print("I don't know that spell (command).")
63.
64.         # 5. End of program
65.         print("System Closed.")

```

اجراکن و لذت ببر! برنامه را اجرا کن. چند قلم جنس (شمشیر، سپر، نقشه گنج) اضافه کن، لیست را ببین، نقشه را حذف کن و بعد سعی کن چیزی که در کوله پشتی نیست را حذف کنی تا ببینی برنامه چطور هوشمندانه خطا را مدیریت می‌نماید.

تبریک می‌گوییم! تو الان ساختار اصلی تمام نرم‌افزارهای دنیا (چرخه دریافت دستور و پردازش) را ساختی. این یک قدم بزرگ بود!

| 🌸 ارتقاء لول: 🌸 |

۱۵ → ۱۰

مهارت‌های جدید:

◆ ساختار List

لِ نام باستانی: گنجینه بی‌انتهای داده.
توضیح: دیگر نیازی به حمل تک‌تک داده‌ها نیست. همه چیز در یک صف منظم است.

◆ متد Append

لِ نام باستانی: به خدمت‌گیری نیرو.
توضیح: افزودن سربازی تازه به ارتش داده‌های تو.

فصل ۶: رازهای کوله‌پشتی حرفه‌ای: تابلوی افتخارات ماجراجویان

مأموریت فصل: ساخت یک تابلوی امتیازات برای بازی حدس عددمان.

نقشهٔ راه:

- برش زدن لیست (Slicing): چگونه فقط چند آیتم خاص را برداریم؟
- کیسهٔ مهر و موم شده (Tuples): آیتم‌های جادویی که تغییر نمی‌کنند؛
- ترکیب جادوها: گشتن روی لیستی از تاپل‌ها؛
- جادوی نظم‌دهی (مرتب کردن لیست)؛
- پروژه: ساخت تابلوی امتیازات و نمایش ۳ نفر برتر.

۱. هنر برش زدن

گاهی ما نمی‌خواهیم فقط یک آیتم را برداریم، بلکه می‌خواهیم بخشی از لیست را جدا کنیم (مثلاً سه نفر اول برای تیم حمله). به این کار برش زدن^۱ می‌گویند.

ساختار کلی طلسم برش به این صورت است:

```
list_name[start : end : step]
```

- **شروع^۲:** از چه ایندکسی شروع کنم؟ (شامل خودش هم می‌شود)؛
- **پایان^۳:** تا کجا بروم؟ (شامل خودش نمی‌شود و یکی قبل از آن می‌ایستد)؛
- **گام^۴:** چندتا چندتا بروم؟ (اگر ننویسید، پیش‌فرض آن ۱ است).

بیایید چند برش جادویی را روی لیست ماجراجویان امتحان کنیم:

```
1. # Our list of adventurers
2. adventurers = ["Aria", "Kael", "Lyra", "Zane", "Finn", "Bora"]
3.
4. # 1. The Standard Slice (Start to End)
5. # Get from index 1 up to (but not including) 4
6. team_alpha = adventurers[1:4]
7. print("Team Alpha:", team_alpha)
8. # Output: ['Kael', 'Lyra', 'Zane']
9.
10. # 2. The Shortcut Magic (Leaving blank)
11. # Empty start = Start from 0
12. first_three = adventurers[:3]
13. print("First Three:", first_three)
14. # Output: ['Aria', 'Kael', 'Lyra']
15.
16. # Empty end = Go to the very end
17. from_third_onwards = adventurers[2:]
18. print("From Lyra to End:", from_third_onwards)
19. # Output: ['Lyra', 'Zane', 'Finn', 'Bora']
```

^۱ Slicing

^۲ Start

^۳ End

^۴ Step

۱-۱. قدم‌های بلند (Step)

پارامتر سوم، یعنی step، به ما اجازه می‌دهد که از روی آیتم‌ها بپریم. مثلاً می‌خواهیم یکی‌درمیان انتخاب کنیم:

```
1. # 3. The Jump Spell (Step = 2)
2. # Start from 0, go to end, take every 2nd person
3. odd_team = adventurers[::2]
4. print("Odd Team:", odd_team)
5. # Output: ['Aria', 'Lyra', 'Finn'] (Index 0, 2, 4)
```

۲-۱. طلسم معکوس زمان (Negative Step)

این جذاب‌ترین بخش ماجراست! اگر برای step از عدد منفی استفاده کنیم، پایتون دنده‌عقب حرکت می‌کند.

عدد ۱- یعنی: از آخر به اول، یکی‌یکی بیا.

```
1. # 4. The Time Reversal Spell (Reversing a list)
2. # Start from end, go to start, step -1
3. reversed_team = adventurers[::-1]
4.
5. print("Reversed Team:", reversed_team)
6. # Output: ['Bora', 'Finn', 'Zane', 'Lyra', 'Kael', 'Aria']
```

خلاصهٔ سریع دستورات برش:

معنی جادویی	دستور
از ۱ تا ۲ (و ۳ حساب نمی‌شود).	<code>list[1:3]</code>
از ۰ تا ۲ (۳ تا ی اول).	<code>list[:3]</code>
از ۲ تا آخر آخر.	<code>list[2:]</code>
کپی کامل از کل لیست.	<code>list[:]</code>
یکی‌درمیان (پرش دوتایی).	<code>list[::2]</code>
کل لیست برعکس می‌شود.	<code>list[::-1]</code>

۲. کیسهٔ مهر و موم شده (Tuples)

لیست‌ها عالی‌اند چون می‌توانیم آیتم‌ها را به آنها `append` یا `remove` کنیم. آنها تغییرپذیر^۵ هستند.

اما گاهی غنائمی داریم که هرگز نباید تغییر کنند، مثل مختصات جادویی یک گنج (`X=10, Y=20`). یا یک رکورد امتیاز (که نام بازیکن و امتیازش به هم قفل شده‌اند).

برای این کار، از تاپل^۶ استفاده می‌کنیم. تاپل‌ها دقیقاً مثل لیست‌ها هستند، اما با پرانتز () ساخته می‌شوند و تغییرناپذیرند^۷ (اگر تلاش به تغییر آنها کنید با خطا مواجه می‌شوید).

```
1. # A list is mutable (can change)
2. mutable_list = [10, 20]
3. mutable_list.append(30) # This works!
4. print("Mutable list:")
5. print(mutable_list)
6.
7. # A tuple is immutable (cannot change)
8. # Tuples are defined using parentheses ( )
9. immutable_tuple = (10, 20)
10. print("Immutable tuple:")
11. print(immutable_tuple)
12.
13. # If you try to change a tuple, you get an ERROR!
14. # immutable_tuple.append(30) # This will crash the code! (AttributeError)
```

ما از تاپل‌ها برای داده‌هایی استفاده می‌کنیم که می‌خواهیم مطمئن باشیم به‌صورت تصادفی تغییر نمی‌کنند.

۳. جادوی باز کردن تاپل (Tuple Unpacking)

حالا که تاپل داریم، چطور به محتویات آن دسترسی پیدا کنیم؟ راه عادی (که از لیست‌ها بلدیم)، استفاده از ایندکس است:

```
1. # A single high score record
2. score_record = ("Aria", 100)
3.
4. # The "old" way using indexes
5. name = score_record[0]
6. score = score_record[1]
7. print(name + " scored " + str(score))
```

^۵ Mutable

^۶ Tuple

^۷ Immutable

این روش کار می‌کند، اما یک راه جادویی‌تر و تمیزتر هم وجود دارد.

پایتون به ما اجازه می‌دهد تا یک تاپل را در یک خط باز کنیم و هر آیتم آن را مستقیماً داخل یک متغیر بریزیم. به این کار باز کردن تاپل^۸ می‌گویند.

```
1. # The "magic" way: Tuple Unpacking
2. score_record = ("Aria", 100)
3.
4. # Python smartly puts "Aria" into 'name' and 100 into 'score'
5. name, score = score_record
6.
7. print(name + " scored " + str(score))
```

هر دو کد بالا یک نتیجه می‌دهند، اما روش دوم بسیار خواناتر و پایتونیک^۹ تر است. ما به جای دو خط کد برای دسترسی به آیت‌ها، در یک خط آنها را باز کردیم.

نکته: همچنین می‌توانیم از این روش برای لیست هم استفاده کنیم.

```
1. my_list = ["mahdi", 200]
2. name, score = my_list
```

۴. ترکیب جادوها: لیستی از تاپل‌ها

خب، جادوی واقعی زمانی اتفاق می‌افتد که این دو را ترکیب کنیم. یک تاپل برای قفل کردن داده‌ها (مثل نام و امتیاز) عالی‌ست، یک لیست برای نگهداری مجموعه‌ای از این تاپل‌ها فوق‌العاده است!

این ساختار، ستون فقرات تابلوی امتیازات ما خواهد بود:

```
1. # Our High Score board
2. # It's a LIST (backpack) containing Tuples (sealed records)
3. high_scores = [
4.     ("Aria", 100),
5.     ("Kael", 80),
6.     ("Lyra", 75),
7.     ("Zane", 60),
8.     ("Finn", 30)
9. ]
```

حالا چطور روی این لیست سرشماری کنیم؟ با حلقه `for`.

^۸ Tuple Unpacking

^۹ Pythonic

اما این بار، می‌توانیم از جادوی باز کردن تاپل (که در بخش قبل یاد گرفتیم) مستقیماً در خود حلقه `for` استفاده کنیم:

```
1. print("--- All Scores (unpacked) ---")
2.
3. # Instead of: for entry in high_scores:
4. # We write:   for name, score in high_scores:
5. # Python unpacks each tuple (like '("Aria", 100)')
6. # into 'name' and 'score' in each loop.
7. for name, score in high_scores:
8.     # Now we use str() to join text and numbers (from Chapter 4)
9.     print(name + " scored " + str(score) + " points")
```

خروجی این کد بسیار زیباتر است:

```
--- All Scores (unpacked) ---
Aria scored 100 points
Kael scored 80 points
...
```

۵. جادوی نظم‌دهی (مرتب کردن لیست)

تا اینجا یاد گرفتیم چطور داده‌ها را در لیست ذخیره کنیم، اما گاهی اوقات کوله‌پشتی ما حسابی شلخته می‌شود و نیاز به یک لیست مرتب‌شده از اعداد داریم.

پایتون یک طلسم جادویی آماده به نام `sort()` دارد که در یک چشم‌برهم‌زدن، همه‌چیز را منظم می‌کند.

۱. از کم به زیاد (حالت پیش‌فرض):

وقتی دستور `sort()` را صدا می‌زنید، پایتون اعداد را از کوچک‌ترین به بزرگ‌ترین می‌چیند.

```
1. # 1. A messy list of scores
2. scores = [50, 10, 100, 5, 20]
3.
4. # 2. The sorting spell
5. scores.sort()
6.
7. # 3. See the result
8. print("Sorted scores:", scores)
```

خروجی:

```
Sorted scores: [5, 10, 20, 50, 100]
```

۲. از زیاد به کم (معکوس):

اما در بازی‌ها، معمولاً کسی که بیشترین امتیاز را دارد اول است. برای این کار، باید یک تنظیم اضافی به دستورمان اضافه کنیم. ما به پایتون می‌گوییم `reverse=True`، یعنی برعکسش کن!

```
1. scores = [50, 10, 100, 5, 20]
2.
3. # Sort from biggest to smallest
4. scores.sort(reverse=True)
5.
6. print("Top scores:", scores)
```

خروجی:

```
Top scores: [100, 50, 20, 10, 5]
```

۶. پروژه: تابلوی افتخارات^{۱۰} – 100 xp

ماجراجو، تو آماده‌ای! در انجمن ماجراجویان، هر روز صدها نفر مأموریت انجام می‌دهند، اما فقط بهترین‌ها روی دیوار افتخارات ثبت می‌شوند. در این مأموریت، می‌خواهیم با استفاده از جادوی تاپل‌ها (بسته‌های غیرقابل‌تغییر) و برش زدن، تابلوی امتیازات بازیمن را بسازیم و ۳ قهرمان برتر را جدا کنیم.

۶-۱. تصویر نهایی: این تابلو چه شکلی است؟

قبل از کدنویسی، خروجی را تصور کن. یک لیست طولانی از بازیکنان داریم. برنامه ما باید دو کار انجام دهد:

۱. اول لیست همه بازیکنان را نشان دهد (تا کسی ناراحت نشود!);

۲. سپس با یک خط جداکننده، فقط ۳ نفر اول را با شماره رتبه (۱، ۲، ۳) به عنوان برندگان نهایی اعلام کند.

خروجی باید چیزی شبیه به این باشد:

```
--- All Scores ---
Aria scored 100 points
Kael scored 80 points
...
--- TOP 3 CHAMPIONS ---
1. Aria scored 100 points
2. Kael scored 80 points
3. Lyra scored 75 points
```

^{۱۰} The High Score Board

۶-۲. نقشه و طرح اولیه

برای مدیریت این داده‌ها، به یک نقشه دقیق نیاز داریم:

۱. ساخت مخزن داده (لیست تاپل‌ها): باید نام بازیکن و امتیاز او را به هم بچسبانیم تا گم نشوند. بهترین راه چیست؟ تاپل ("Name", Score). سپس همه تاپل‌ها را داخل یک لیست می‌ریزیم.

- **منطق:** یک لیست می‌سازیم که داخلش ۵ تاپل وجود دارد. فرض می‌کنیم لیست از قبل مرتب شده است (درمورد مرتب کردن لیستی از تاپل‌ها بعداً صحبت می‌کنیم).

۲. اعلام همگانی (Unpacking): می‌خواهیم لیست را چاپ کنیم. اما نمی‌خواهیم خروجی زشت باشد، مثل ('Aria', 100).

- **منطق:** از یک حلقه for استفاده می‌کنیم. اما اینجا تکنیک باز کردن تاپل را به کار می‌بریم. یعنی در همان خط اول حلقه، اسم و امتیاز را از داخل تاپل بیرون می‌کشیم و در دو متغیر جدا می‌ریزیم.

۳. جدا کردن قهرمانان (Slicing): ما فقط ۳ نفر اول را می‌خواهیم.

- **منطق:** لازم نیست حلقه جدیدی بنویسیم یا بشماریم. از قابلیت برش زدن استفاده می‌کنیم تا از خانه ۰ تا ۳ لیست را بگیریم و در یک لیست جدید (top_3) بریزیم.

۴. مراسم اهدای جوایز (Rank Loop): حالا لیست ۳ نفره را چاپ می‌کنیم، اما این باریک شمارنده هم کنارش می‌گذاریم تا معلوم شود نفر اول کیست.

۶-۳. چالش‌های فکری

قبل از اینکه کد نهایی را ببینی، سعی کن این معماها را حل کنی:

- **چالش ۱. تاپل یا لیست؟:** چرا برای نگه داشتن (نام و امتیاز) یک نفر، از تاپل () استفاده می‌کنیم و نه لیست [] ؟ (راهنمایی: چون نام و امتیاز یک بازیکن یک زوج جدانشدنی‌اند و نباید تغییر کنند).

- **چالش ۲. فرمول برش:** اگر بخواهیم ۳ نفر اول را برداریم، فرمول برش list[start:stop] چه می‌شود؟ (راهنمایی: از ۰ شروع کن و سر ۳ متوقف شو. پایتون عدد آخر (Stop) را شامل نمی‌شود، پس در نتیجه داریم: آیتم ۰، آیتم ۱ و آیتم ۲).

Commented [GG17]: منطق کلمه درستی نیست اینجا. راهکار یا روش بهتر نیست؟

Commented [GG18]: عددهای این پاراگراف نباید انگلیسی و با فونت کد باشن؟

- چالش ۳. شمارندهٔ رتبه: در حلقهٔ `for`، ما به متغیرهای لیست دسترسی داریم، اما چطور بفهمیم نفر چندم است؟ (راهنمایی: باید یک متغیر، مثل `rank = 1` قبل از حلقه بسازی و در هر دور حلقه، یکی به آن اضافه کنی).

۶-۴. کد نهایی پروژه (The Spell)

وقت آن است که تابلوی افتخارات را بسازیم. کد زیر را در فایل `project.py` بنویس. به نحوهٔ باز کردن تاپل‌ها در حلقهٔ `for` خیلی دقت کن.

```
1. # --- The High Score Board ---
2.
3. # 1. Setup the data
4. # We use a LIST of TUPLES.
5. # Each tuple is (Name, Score) and creates a solid, unchangeable pair.
6. high_scores = [
7.     ("Aria", 100),
8.     ("Kael", 80),
9.     ("Lyra", 75),
10.    ("Zane", 60),
11.    ("Finn", 30)
12. ]
13.
14. print("--- All Scores ---")
15.
16. # 2. Display all scores using "Tuple Unpacking"
17. # Instead of getting one item, we get "name" and "score" directly!
18. for name, score in high_scores:
19.     # Remember to convert score (int) to string (str) for printing
20.     print(name + " scored " + str(score) + " points")
21.
22. print("") # An empty print() adds a blank line for better look
23. print("--- TOP 3 CHAMPIONS ---")
24.
25. # 3. Get the Top 3 using "Slicing"
26. # Logic: Start at index 0, Stop BEFORE index 3 (so we get 0, 1, 2)
27. top_3_list = high_scores[0:3]
28.
29. # 4. Display the Top 3 with ranking
30. rank = 1 # We start ranking from 1
31.
32. for name, score in top_3_list:
33.     # We print the Rank, Name, and Score all together
34.     print(str(rank) + ". " + name + " scored " + str(score) + " points")
35.
36.     # Increase rank for the next champion
37.     rank = rank + 1
```

Commented [GG19]: کامنت ناشر: کاش دلیل این تبدیل و اگر فراموش کنیم چه اروری رخ می‌دهد هم توضیح داده می‌شد

اجرا کن و نتیجه را ببین! خروجی باید دقیقاً لیست کامل و سپس لیست ۳ نفر برتر را نشان دهد.

۵-۶. جشن پیروزی و تحلیل

آفرین! تو همین الان دو مورد از قدرتمندترین تکنیک‌های مدیریت داده در پایتون را یاد گرفتی:

۱. **Tuples**: یاد گرفتی چگونه داده‌های مرتبط (مثل نام و امتیاز) را در بسته‌های امن نگه داری.

۲. **Slicing**: یاد گرفتی چگونه مثلاً یک نینجا، بخشی از یک لیست بزرگ را برش بزنی و استخراج کنی.

کوله‌پشتی تو حالا واقعاً حرفه‌ای شده‌است. این تکنیک‌ها در آینده برای کار با داده‌های بزرگ بسیار حیاتی‌اند!

| 🌸 ارتقاء لول: 🌸 |

۱۵ → ۱۸

مهارت‌های جدید:

🔹 تکنیک Slicing

له نام باستانی: هنر برش لیزری.

توضیح: جدا کردن بخشی از کل. انتخاب دقیق آنچه نیاز داری از دل انبوه اطلاعات.

🔹 ساختار Tuple

له نام باستانی: لوح‌های سنگی.

توضیح: داده‌هایی که حک شده‌اند و تغییرناپذیرند.

فصل ۷: کتاب جادویی: ساخت مترجم کلمات

مأموریت فصل: ساخت یک دیکشنری ساده برای ترجمه کلمات در سفرهای خارجی.

نقشه راه:

- دیکشنری‌ها (Dictionaries): کتاب جادویی که داده‌ها را به صورت (کلید : مقدار) ذخیره می‌کند؛
- جادوی دیکشنری: دسترسی، اضافه کردن، تغییر دادن و حذف کردن کلمه‌ها؛
- تله کلید گمشده (The KeyError): اگر کلمه‌ای در کتاب نباشد چه اتفاقی می‌افتد؟
- طلسم‌های ایمنی (get و in): دو راه برای جلوگیری از ارور در زمان بازیابی؛
- سرشماری در کتاب جادویی: چطور روی تمام کلمات و معانی آنها حلقه بزنیم؟
- پروژه: ساخت مترجمی که چند کلمه فارسی را به انگلیسی برمی‌گرداند.

Commented [GG20]: بهتره توی خط جدا بیاد یا کلا حذف بشه چون به فارسی نوشته شده ممکنه غلط انداز باشه. در صورتی که نحوه‌ی درستش اینکه کلید سمت چپ و مقدار سمت راست باشد یا کلا حذف بشه یا یک جوری اصلاح بشه که کسی رو به اشتباه نندازه- یا میتونه به صورت «(کلید-مقدار)» نوشته بشه

۱. دیکشنری‌ها: کتابچه راهنمای هوشمند

تابه حال، ما از لیست ([]) برای نگهداری آیتم‌ها استفاده می‌کردیم، درست مثل کوله‌پشتی‌ای که وسایل را پشت سر هم داخلش می‌انداختیم. در لیست، ترتیب مهم بود و برای پیدا کردن اولین وسیله باید می‌گفتیم: «شمارهٔ ۰ را بده».

اما نوع جدیدی از ساختار داده به نام دیکشنری^۱ وجود دارد که با آکولاد { } ساخته می‌شود. دیکشنری شبیه یک قفسه با برچسب‌هایی برای هر خانه‌ست. اینجا دیگر شماره‌ها (0, 1, 2) مهم نیستند، برچسب‌ها مهم‌اند.

در دیکشنری، اطلاعات به صورت جفت‌های کلید-مقدار (Key : Value) ذخیره می‌شوند:

- **کلید (Key):** مثل **برچسب** روی یک خانه از قفسه است. کلمه‌ای که با آن دنبال چیزی می‌گردیم (باید منحصر به فرد و غیر تکراری باشد، یعنی نمی‌شود دو قفسه با برچسب یکسان داشت).
- **مقدار (Value):** چیزی است که داخل آن قفسه قرار دارد (معنی کلمه یا دادهٔ اصلی).

۱-۱. مثال اول: مترجم جیبی (واژه‌نامهٔ انگلیسی به فارسی)

بیایید ساده‌ترین کاربرد آن را ببینیم. دقیقاً مثل یک واژه‌نامهٔ واقعی! ما کلمهٔ انگلیسی را می‌دهیم و معنی فارسی را می‌گیریم. (با قاعدهٔ نوشتاری کلید دونقطه مقدار. اگر چند کلید-مقدار داشتیم، با کاما (,) آنها را جدا می‌کنیم).

```
1. # A simple English to Persian dictionary
2. my_translator = {
3.     "apple": "سیب",
4.     "water": "آب",
5.     "book": "کتاب",
6.     "friend": "دوست"
7. }
8.
9. # How to use it:
10. print(my_translator["apple"]) # Output: سیب
11. print(my_translator["book"]) # Output: کتاب
```

می‌بینید؟ نگفتیم ایندکس شمارهٔ ۰ یا ۱ را بده، بلکه گفتیم: «معنی apple چیست؟» و او جواب داد.

^۱ Dictionary

^۲ Dictionary

۲-۱. مثال دوم: کارت شناسایی ماجراجو

حالا بیا ببینیم این را در پروژه خودمان ببینیم. اگر بخواهیم مشخصات قهرمان داستان را ذخیره کنیم، استفاده از لیست گیج‌کننده است (یادمان می‌رود که اسم در خانه دوم بود یا سوم). اما با دیکشنری همه چیز روشن است:

```
1. # We use curly braces {} for dictionaries
2. # We use colons (:) to link a KEY to a VALUE
3. adventurer_stats = {
4.     "name": "Kael",
5.     "level": 5,
6.     "class": "Mage",
7.     "gold": 150
8. }
```

حالا جادوی واقعی: برای دسترسی به اطلاعات، ما دیگر کورکورانه از ایندکس [0] استفاده نمی‌کنیم. مستقیماً کلید را صدا می‌زنیم تا مقدارش ظاهر شود:

```
1. # Accessing data using the 'key'
2. print("Adventurer's Name:")
3. print(adventurer_stats["name"]) # Output: Kael
4.
5. print("Current Level:")
6. print(adventurer_stats["level"]) # Output: 5
```

این روش بسیار خواناتر و حرفه‌ای‌تر از حدس زدن شماره‌هاست! انگار به جای گشتن در جیب‌های کوله‌پشتی، دقیقاً می‌دانیم هر چیزی کجاست.

۲. جادوی دیکشنری: افزودن، تغییر و حذف

قدرت دیکشنری‌ها در اضافه کردن، تغییر دادنِ خیلی راحت و جست‌وجوی سریع است.

۲-۱. اضافه کردن یا تغییر دادن یک کلمه

برخلاف لیست‌ها که برای اضافه کردن، نیاز به `append` داشتند، دیکشنری‌ها مستقیماً عمل می‌کنند.

قانون طلایی این است: کلید را صدا بزن و مقدار را به آن بده.

این دستور هوشمندانه عمل می‌کند:

۱. اگر کلید جدید باشد، آن را به دیکشنری اضافه می‌کند.

Commented [GG22]: کامنت ناشر: از این لحن ممکنه برداشت بشه که دیکشنری‌ها بهتر از لیست‌ها هستند. احتمالاً اگه کاربردشون رو توی دنیای واقعی مثال بزنیم و تفاوت‌ها و مزیت‌های هر کدوم رو یادآور بشیم بهتر باشد

۲. اگر کلید از قبل وجود داشته باشد، مقدار قبلی را پاک کرده و مقدار جدید را جایگزین^۳ می‌کند.

بیایید این جادو را در عمل ببینیم:

```
1. # Our translator starts empty
2. translator = {}
3.
4. # Add a new word (key-value pair)
5. translator["hello"] = "salam"
6. translator["goodbye"] = "khodahafez"
7.
8. print("Translator after adding words:")
9. print(translator) # Output: {'hello': 'salam', 'goodbye': 'khodahafez'}
10.
11. # Change an existing word (update the value)
12. translator["hello"] = "dorood"
13. print("Translator after update:")
14. print(translator) # Output: {'hello': 'dorood', 'goodbye': 'khodahafez'}
```

۲-۲. حذف کردن یک کلید

اگر بخواهیم یک کلمه و معنی آن را کاملاً از کتاب جادویی پاک کنیم، از طلسم **del** (مخفف delete به معنی حذف) استفاده می‌کنیم:

```
1. translator = {"hello": "salam", "goodbye": "khodahafez"}
2. print("Translator before delete:")
3. print(translator)
4.
5. # Delete the word 'hello'
6. del translator["hello"]
7.
8. print("Translator after delete:")
9. print(translator) # Output: {'goodbye': 'khodahafez'}
```

۳. تله کلید گم‌شده (The KeyError)

هشدار ماجراجو! یک تله بزرگ در دیکشنری‌ها پنهان شده‌است. چه اتفاقی می‌افتد اگر سعی کنی به کلمه‌ای دسترسی پیدا کنی که در کتاب جادویی تو وجود ندارد؟

```
1. translator = {"hello": "dorood"}
2. # Try to access a key that DOES NOT exist
3. print(translator["cat"])
```

^۳ Update

پایتون ارور می‌دهد: `KeyError: 'cat'`. این یک باگ جدید است. پایتون می‌گوید که: «(من کلیدی به نام 'cat' در این کتاب پیدا نمی‌کنم!)».

۴. طلسم‌های ایمنی (in و get)

برای جلوگیری از ارور کلید گم‌شده، می‌توانیم از ۲ طلسم استفاده کنیم:

۴-۱. طلسم اول: بررسی با in

می‌توانیم از همان طلسم `in` (که برای لیست‌ها یاد گرفتیم) استفاده کنیم تا ببینیم آیا کلید (کلمه) در دیکشنری ما وجود دارد یا نه.

```
1. translator = {"hello": "dorood"}
2. word_to_find = "cat"
3.
4. # Check if the 'key' is in the dictionary
5. if word_to_find in translator:
6.     print("Translation is:")
7.     print(translator[word_to_find])
8. else:
9.     print("Sorry, that word was not found.") # This will run!
```

این امن‌ترین و رایج‌ترین راه است.

۴-۲. طلسم دوم (حرفه‌ای): متد get()

دیکشنری‌ها یک طلسم جادویی داخلی به نام `get()` دارند. این طلسم دو ورودی می‌گیرد:

۱. کلیدی که دنبالش می‌گردی؛

۲. یک مقدار پیش‌فرض که اگر کلید پیدا نشد، آن را برگرداند. (این ورودی اختیاری است و اگر آن را وارد نکنیم، پیش‌فرضی معادل `None`، به معنای هیچ، برمی‌گرداند).

این طلسم هرگز کرش^۴ نمی‌کند!

```
1. translator = {"hello": "dorood"}
2.
3. # Get the translation for 'hello'
4. translation_1 = translator.get("hello")
5. print(translation_1) # Output: dorood
6.
7. # Get the translation for 'cat'
8. translation_2 = translator.get("cat", "Word not found.")
```

^۴ Crash

```
9. print(translation_2) # Output: Word not found. (No crash!)
```

Commented [GG23]: بهتره در صفحه قبل بیاد

۵. سرشماری در کتاب جادویی (Looping)

قبلاً با حلقه `for` روی لیست‌ها گشته‌ایم. حالا چطور می‌توانیم روی دیکشنری حلقه بزنیم؟

۵-۱. حلقه زدن روی کلیدها

اگر به‌سادگی روی دیکشنری، حلقه `for` بزنی، پایتون کلیدها را به تو می‌دهد:

```
1. translator = {"hello": "dorood", "goodbye": "khodāhāfez"}
2.
3. print("--- Dictionary Keys ---")
4. for word in translator:
5.     print(word)
```

خروجی:

```
--- Dictionary Keys ---
hello
goodbye
```

۵-۲. حلقه زدن روی مقادارها

اگر فقط به معانی (مقادارها) نیاز داری، از طلسم `values()` استفاده کن:

```
1. print("--- Dictionary Values ---")
2. for meaning in translator.values():
3.     print(meaning)
```

خروجی:

```
--- Dictionary Values ---
dorood
khodāhāfez
```

۵-۳. حلقه زدن روی هر دو (Key & Value) – بهترین روش

و اما قدرتمندترین جادو! اگر بخواهی هم کلید و هم مقدار را با هم داشته باشی، از طلسم `items()` استفاده کن. این طلسم لیستی از تاپل‌ها را برمی‌گرداند!

می‌توانیم از همان جادوی بازکردن تاپل در حلقه `for` استفاده کنیم:

```
1. print("--- Full Translator ---")
2. # .items() gives us (key, value) pairs
3. for word, meaning in translator.items():
4.     print(word + " means " + meaning)
```

خروجی:

```
--- Full Translator ---
hello means dorood
goodbye means khodāhāfez
```

۶. هشدار ماجراجو: طلسم نمایش حروف فارسی

یک تله کوچک ممکن است در مسیر ماجراجویی تو وجود داشته باشد! گاهی اوقات، اتاق فرمان (ترمینال) در برخی سیستم‌ها (به خصوص نسخه‌های قدیمی‌تر ویندوز) نمی‌داند چطور با متن‌های فارسی کار کند.

اگر بعد از وارد کردن یک کلمه فارسی، در خروجی علامت سؤال (?????) یا حروف عجیب و غریب دیدی، نترس! این یعنی ترمینال تو به یک طلسم جادویی نیاز دارد تا زبان فارسی را بفهمد.

اگر در ترمینال VS Code یا ویندوز به مشکل خوردی، قبل از اجرای برنامه، این دستور را یک بار در ترمینال تایپ کن و Enter را بزن (به ازای هربار باز کردن ترمینال، نیازست مجدداً این دستور را اجرا کنی):

```
chcp 65001
```

این طلسم به ویندوز می‌گوید که از کتاب رمزگشایی استاندارد جهانی (UTF-8) برای خواندن حروف استفاده کند. در اکثر سیستم‌های مدرن (مثل macOS، لینوکس و Windows Terminal جدید) این مشکل وجود ندارد، اما دانستن این طلسم، تو را برای هر چالشی آماده می‌کند!

نکته: اگر فارسی درست نمایش داده نمی‌شود، حتی با اینکه قبل از اجرای کد، دستور `chcp 65001` را داخل ترمینال زدی، ناراحت نشو. کاملاً طبیعی‌ست و همیشه فارسی در ترمینال مشکل‌سازست.

۷. پروژه: ساخت مترجم هوشمند کلمات - 100 xp

ماجراجو، تو آماده‌ای!

در فصل‌های قبل یاد گرفتیم چطور داده‌ها را در لیست ذخیره کنیم. اما اگر بخواهیم معنی کلمات را پیدا کنیم، گشتن در یک لیست طولانی خیلی سخت و کند است.

امروز با استفاده از ابزار جدید و قدرتمندمان، یعنی دیکشنری، یک مترجم انگلیسی‌به‌فارسی می‌سازیم.

Commented [GG24]: منظور از کتاب رمزگشایی چیه؟ لازم نیست معادل انگلیسی‌شو بدین؟

Commented [GG25]: خب؟ همین؟ راهکار دیگه‌ای نباید به خواننده داد؟

۷-۱. تصویر نهایی: این برنامه قرار است چه کار کند؟

قبل از کدنویسی، بیا نحوه کار این مترجم را ببینیم. می‌خواهیم برنامه‌ای بسازیم که مثل یک واژه‌نامه جیبی هوشمند عمل کند:

۱. از تو می‌پرسد: چه کلمه‌ای را ترجمه کنم؟

۲. اگر کلمه‌ای که بلد باشد (مثلاً Hello) را بنویسی: سریع جواب می‌دهد: «سلام».

۳. اگر کلمه‌ای که بلد نیست (مثلاً Dragon) را بنویسی: می‌گوید: «شرمنده، این کلمه در دایره لغات من نیست».

۴. و این کار را تکرار می‌کند تا زمانی که بنویسی quit.

۷-۲. نقشه و طرح اولیه

بیایید منطق این ابزار را مهندسی کنیم. ما به ۳ بخش اصلی نیاز داریم:

۱. حافظه (The Memory): ما به یک حافظه نیاز داریم که کلمات انگلیسی و معنی فارسی آنها را جفت جفت نگه دارد.

- منطق: از یک دیکشنری { } استفاده می‌کنیم. کلمه انگلیسی، کلید (Key) و کلمه فارسی مقدار (Value) می‌شود.

۲. حلقه بی‌پایان (The Loop): مترجم نباید بعد از یک کلمه بسته شود.

- منطق: یک حلقه while True می‌سازیم که تا ابد بچرخد (مگر اینکه خودمان متوقفش کنیم).

۳. موتور جست‌وجو (The Search Engine): وقتی کاربر کلمه‌ای را وارد کرد، باید در پایگاه داده بگردیم.

- منطق:

- اول چک کن کاربر quit نوشته‌است یا نه (برای فرار از حلقه)؛

- دوم چک کن آیا کلمه در دیکشنری هست؟ (با دستور in)؛

- اگر بود: کلید را بده، مقدار را بگیر (dict[key])؛

- اگر نبود: پیام خطا بده.

Commented [GG26]: به نظرم باید برعکس باشه. چون قدمای

قبلی رو «ماشین» داره بررسی می‌کنه. در این قدم هم ماشین باید

کلمه انگلیسی (کلید) را بگیرد و ترجمه (مقدار) را بدهد. پس فاعل

«ماشین» است. یا باید ماشین بگوید «کلید را بده، مقدار را بگیر». یا

ماشین کلید را «میگیرد» و مقدار را «می‌دهد»

۳-۷. چالش‌های فکری

قبل از اینکه کد را ببینی، سعی کن به این سؤالات پاسخ بدهی:

- چالش ۱. ساختار دیکشنری: یادت هست فرق دیکشنری با لیست چه بود؟ لیست‌ها با [] و اعداد (ایندکس) کار می‌کردند. دیکشنری‌ها با چه علامتی ساخته می‌شدند؟ (راهنمایی: آکولاد { }).
- چالش ۲. کلید یا مقدار؟: ما می‌خواهیم کلمه انگلیسی را بگیریم و فارسی را تحویل دهیم. پس کلمه انگلیسی باید Key باشد یا Value؟ (راهنمایی: چیزی که جست‌وجو می‌کنیم همیشه Key است).
- چالش ۳. حساسیت به حروف: پایتون خیلی دقیق است. از نظر پایتون "Hello" با "hello" فرق دارد. در این پروژه ساده، ما کلمات را دقیقاً همان‌طور که در دیکشنری نوشتیم وارد می‌کنیم (مثلاً با حرف بزرگ).

۴-۷. کد نهایی پروژه (The Spell)

وقت نوشتن است. کد زیر را در فایل `project.py` بنویس. دقت کن که چطور کلمات انگلیسی (سمت چپ) و فارسی (سمت راست) با دونقطه : به هم وصل شده‌اند.

Commented [GG27]: کامنت ناشر: چگونه که هر پروژه اسم مخصوص به خودش رو داشته باشه؟

```
1. # --- The English to Farsi Translator ---
2.
3. # 1. Setup the Dictionary (Database)
4. # Structure -> Key (English) : Value (Farsi)
5. english_to_farsi = {
6.     "Hello": "سلام",
7.     "Goodbye": "خداحافظ",
8.     "Cat": "گربه",
9.     "Dog": "سگ",
10.    "House": "خانه",
11.    "Coder": "برنامه‌نویس"
12. }
13.
14. print("Welcome to the Smart Translator!")
15. print("--- Type 'quit' to exit ---")
16.
17. # 2. Start the main loop (Infinite Loop)
18. # We use 'while True' to keep running forever until 'break'
19. while True:
20.     print("-----")
21.
22.     # Get input from the user
23.     word = input("Enter an English word to translate: ")
24.
25.     # 3. Check for exit command FIRST
26.     if word == "quit":
27.         print("Goodbye, Adventurer!")
```

```

28.         break # The spell to smash the loop and exit!
29.
30.     # 4. Search logic (Safety Spell)
31.     # check IF the word exists inside the dictionary keys
32.     if word in english_to_farsi:
33.         # Get the translation
34.         translation = english_to_farsi[word]
35.         print("Meaning: " + translation)
36.     else:
37.         # If the word is NOT in the dictionary
38.         print("Sorry, I don't know that word yet.")

```

اجرا کن و نتیجه را ببین!

۱. برنامه را اجرا کن.

۲. کلمه Cat را بنویس. باید ترجمه را ببینی.

۳. کلمه dragon را بنویس. باید پیام نمی‌دانم را ببینی.

۴. حالا کلمه cat (با حروف کوچک) را امتحان کن. چه شد؟ پیدا نشد! (چون در دیکشنری ما Cat با C بزرگ ذخیره شده و پایتون به بزرگ و کوچک بودن حروف حساس است).

۵. در نهایت quit را بنویس و خارج شو.

۷-۵. جشن پیروزی و تحلیل

آفرین!

تو الان سریع‌ترین ابزار جست‌وجوی خودت را ساختی. تو یاد گرفتی که:

- لیست‌ها برای وقتی مناسبند که ترتیب مهم است.
- دیکشنری‌ها برای وقتی عالی‌اند که می‌خواهیم چیزی را سریع پیدا کنیم (مثل دفتر تلفن یا واژه‌نامه).

کوله‌پشتی تو حالا یک مترجم هوشمند دارد که در سرزمین‌های ناشناخته به کارت می‌آید!

Commented [GG28]: مثل خط پایین، نیاز به یک مثال واقعی دارد.

| ارتقاء لول: |

۱۸ → ۲۲

مهارت‌های جدید:

◆ ساختار Dictionary

له نام باستانی: کتاب الاسرار.

توضیح: هر کلیدی، دری را باز می‌کند. سریع‌ترین راه برای یافتن معنای هر چیز.

◆ متد Get

له نام باستانی: دست ایمن.

توضیح: جست‌وجو در کتاب بدون ترس از خطا و سقوط.

فصل ۸: ساخت جادوهای شخصی: هنر ساخت توابع

مأموریت فصل: بازسازی کدی که هی تکرار شده و ساخت یک کارت معرفی جادویی و قابل استفاده مجدد با استفاده از توابع.

نقشه راه:

- یک بار بنویس، صد بار استفاده کن: معرفی توابع (Functions) و اصل DRY؛
- چگونه یک بلوک جادویی را تعریف و آن را صدا بزنیم؟ طلسم‌های `def` و `()`؛
- اولین طلسم ما: ساخت یک تابع ساده برای جلوگیری از تکرار؛
- ورودی توابع (Arguments): چگونه به طلسم خود داده پاس بدهیم؟
- خروجی توابع (Return): چگونه از طلسم خود نتیجه پس بگیریم؟
- جادوی دقیق: برچسب زدن به مواد اولیه؛
- پروژه: رفع طلسم یک کد تکراری برای نمایش مشخصات ماجراجویان.

۱. یک بار بنویس، صد بار استفاده کن: معرفی توابع

در ماجراجویی‌هایمان تا به حال، از طلسم‌های جادویی داخلی پایتون مثل `print()`، `input()`، `len()` و `int()` استفاده کرده‌ایم. اینها توابع^۱ از پیش ساخته شده‌اند.

اما قدرت واقعی آنجاست که خودت تابع مخصوص خودت را بنویسی. یک تابع، یک بلوک کد نام‌گذاری شده و قابل استفاده مجدد است که برای انجام یک کار خاص طراحی شده است.

یک مثال ساده: فکر کن می‌خواهی یک ساندویچ درست کنی. کدنویسی بدون تابع (کاری که تا الان می‌کردیم):

- نان را بردار؛
- کره بمال؛
- پنیر بگذار؛
- نان دوم را بگذار.

(حالا می‌خواهی ساندویچ دوم را درست کنی)

- نان را بردار؛
- کره بمال؛
- پنیر بگذار؛
- نان دوم را بگذار.

۱-۱. کدنویسی با تابع (کاری که یاد می‌گیریم)

دستورالعمل (تابع) می‌سازیم؛ اسمش را می‌گذاریم `make_sandwich()`. داخل این دستورالعمل، چهار مرحله بالا را می‌نویسیم. حالا هر وقت گرسنه شدیم:

- `make_sandwich()` را صدا می‌زنیم (ساندویچ اول)؛
- `make_sandwich()` را دوباره صدا می‌زنیم (ساندویچ دوم).

Commented [GG29]: کامنت ناشر: بهتره یه جایی همینجاها راجع به فراخوانی تابع / صدا زدن تابع صحبت کنیم و معادل انگلیسی فراخوانی / صدا زدن که Call می‌شه هم در پاورقی بیاید.

^۱ Functions

شعار ماجراجویان حرفه‌ای این است: **DRY: Don't Repeat Yourself** یا خودت را تکرار نکن. اگر کدی را دوبار کپی/پیست کردی، احتمالاً باید آن را به یک تابع تبدیل کنی.

۲. تعریف و صدا زدن یک تابع

ساختن یک جادوی شخصی دو مرحله دارد:

- **مرحله ۱: تعریف تابع با def.** اول، باید جادو را به پایتون یاد بدهی. ما این کار را با **def** (مخفف Define به معنی تعیین کردن) انجام می‌دهیم.

```
1. # --- 1. Defining the Spell (Writing the Scroll) ---
2. # We create a new function called 'greet_adventurer'
3. # The code INSIDE (indented) is the spell's logic.
4. def greet_adventurer():
5.     print("Greetings, mighty adventurer!")
6.     print("Welcome to the Python Guild.")
```

اگر این کد را به تنهایی اجرا کنی، هیچ اتفاقی نمی‌افتد! چرا؟ چون ما فقط **دستورالعمل** ساندویچ را نوشته‌ایم، هنوز ساندویچی درست نکرده‌ایم.

- **مرحله ۲: تعریف تابع باعث اجرا شدن آن نمی‌شود!** برای اجرا شدن باید نام تابع را به همراه پرانتز بنویسی، مثل **my_func()**. این طوری تابع تو اجرا می‌شود. (این کار را هرچند بار که دوست داشته باشی می‌توانی انجام دهی).

```
1. # --- 1. Defining the Spell ---
2. def greet_adventurer():
3.     print("Greetings, mighty adventurer!")
4.     print("Welcome to the Python Guild.")
5.
6. # --- 2. Calling the Spell (Making the sandwich) ---
7. print("The guild master steps forward...")
8. greet_adventurer() # <-- The magic happens here!
9.
10. print("The quest begins...")
11. greet_adventurer() # <-- We can call it again, easily!
```

اجرا کن و نتیجه را ببین!

```
The guild master steps forward...
Greetings, mighty adventurer!
Welcome to the Python Guild.
The quest begins...
Greetings, mighty adventurer!
Welcome to the Python Guild.
```

ما یک بار کد را نوشتیم و دو بار (یا صد بار!) از آن استفاده کردیم.

Commented [GG30]: کامنت ناشر: چرا به عنوان مثال از تابع `greet_adventure()` استفاده نشد، احتمالاً بخاطر اینکه نمونه‌اش پایین هست بهتره از این استفاده بشه.

۳. اولین تابع ما: خداحافظی با تکرار

یادت هست در پروژه‌های قبلی مدام از `print("=====")` برای جداسازی استفاده می‌کردیم؟ بیایید این کد تکراری را تمیز کنیم.

```
1. # --- The Old, Repetitive Way ---
2. print("--- Adventurer 1 ---")
3. print("Name: Aria")
4. print("=====")
5. print("--- Adventurer 2 ---")
6. print("Name: Kael")
7. print("=====")
8.
9. # --- The New, Clean Way (with a Function) ---
10.
11. # 1. Define the spell once
12. def print_divider():
13.     print("=====")
14.
15. # 2. Call it whenever we need it
16. print("--- Adventurer 1 ---")
17. print("Name: Aria")
18. print_divider()
19.
20. print("--- Adventurer 2 ---")
21. print("Name: Kael")
22. print_divider()
```

کد جدید تمیزتر، خواناتر و حرفه‌ای‌تر است! (شاید بگویی منطقی نیست که یک خط کد را تبدیل به تابع دوخطی بکنیم. حرفت هم کاملاً درست است، ولی فکر کن اگر بخواهیم این جداکننده^۲ را عوض کنیم، فقط توی یک خط تغییرش می‌دهیم و تغییر همه‌جا اعمال می‌شود).

۴. ورودی و خروجی توابع

طلسم‌های ما خوب، اما کمی خنگ بودند. `greet_adventurer` به همه ماجراجویان یک پیام تکراری داده و `print_divider` فقط یک کار ثابت انجام می‌دهد. چطور طلسم‌هایی بسازیم که بتوانند داده بگیرند و نتیجه (غنیمت) برگردانند؟

^۲ Divider

۱-۴. ورودی تابع

بیا باید تابع را شبیه به یک دستگاه مخلوطکن جادویی تصور کنیم. این دستگاه همیشه یک کار تکراری انجام نمی‌دهد؛ بلکه می‌توانیم به آن مواد اولیه بدهیم تا نتیجه‌ای متفاوت تولید کند. در دنیای برنامه‌نویسی، به این ورودی‌ها پارامتر^۳ و آرگومان^۴ می‌گوییم.

۲-۴. تفاوت ظریف: پارامتر یا آرگومان؟

شاید این دو کلمه شبیه به هم به نظر برسند، اما تفاوتشان مثل تفاوت جای خالی و محتوا است. بیا باید یک مثال ساده بزنیم:

۱. پارامتر: مثل جای باتری در کنترل تلویزیون است. وقتی کارخانه کنترل را می‌سازد، آن جای خالی را طراحی می‌کند تا مشخص شود باتری باید کجا قرار بگیرد (در زمان ساختن تابع).

۲. آرگومان: مثل خودِ باتری است. آن چیزی است که شما واقعاً داخل دستگاه می‌گذارید تا کار کند (در زمان استفاده از تابع).

به زبان‌کد: وقتی تابع را تعریف می‌کنید، اسم متغیر پارامتر است. وقتی تابع را صدا می‌زنید، داده واقعی آرگومان است.

بیا باید درکد زیر ببینیم این جادو چطور کار می‌کند:

```
1. # 'name' is a PARAMETER (a magic box inside the function)
2. def greet_by_name(name):
3.     # 'name' acts like a variable that exists ONLY inside this function
4.     print("Greetings, " + name + "!")
5.     print("Welcome to the guild.")
6.
7. # Now, when we call it, we provide an ARGUMENT (the loot)
8. # An argument is the *actual data* we pass in.
9. greet_by_name("Aria") # "Aria" is put into the 'name' box
10. greet_by_name("Kael") # "Kael" is put into the 'name' box
```

خروجی:

```
Greetings, Aria!
Welcome to the guild.
Greetings, Kael!
Welcome to the guild.
```

^۳ Parameter

^۴ Argument

۳-۴. نتیجه نهایی: طلسم `return`

تابع در ساده‌ترین حالت، مثل یک دستگاه آبمیوه‌گیری است. ما میوه (ورودی) می‌دهیم و انتظار داریم آبمیوه (خروجی) تحویل بگیریم.

کلمه کلیدی **return** همان جایی است که آبمیوه از دستگاه خارج شده و به لیوان شما می‌ریزد. این فرمان تعیین می‌کند که حاصل نهایی کار تابع چه چیزی است.

وظیفه اصلی `return`، مشخص کردن نتیجه نهایی و قابل استفاده تابع است.

○ تعریف تابع آبمیوه‌گیری

```
1. def make_juice(fruit_name):
2.     return fruit_name + " Juice"
```

`return` دو کار حیاتی را هم‌زمان انجام می‌دهد:

- **تحويل محصول:** مقدار محاسبه شده را به بیرون می‌فرستد؛
- **توقف فوری:** اجرای تابع در همان لحظه متوقف می‌شود و پایتون فوراً به همان خطی برمی‌گردد که تابع در آن فراخوانی شده بود.

○ مثال عملی: جریان اجرای برنامه

ببینید چطور برنامه در حین اجرا، موقتاً به تابع می‌رود و سپس دقیقاً به همان خط برمی‌گردد:

```
1. # --- Example: Execution Flow ---
2.
3. # 1. The program starts here
4. print("--- 1. Start Program ---")
5.
6. # The program stops at this line and enters the function.
7. apple_juice = make_juice("Apple") # The function is called here.
8.
9. # 2. The function executes, reaches 'return', stops immediately, and
   returns the value "Apple Juice".
10. # 3. Execution returns exactly to this spot, and the returned value is
    stored in the 'apple_juice' variable.
11.
12. print("--- 2. Return to Main Line ---")
13.
14. # Now we can use the final result.
15. print("My drink is:", apple_juice)
16.
17. # Output: My drink is: Apple Juice
```

Commented [GG31]: خطوط به هم ریخته. بررسی بشه

شرح فرآیند: وقتی پایتون به خط `apple_juice = make_juice("Apple")` می‌رسد، می‌داند که باید فعلاً اجرای این خط را نگه دارد، به سراغ تابع `make_juice` برود، آن را اجرا کند و منتظر بماند تا آبمیوه نهایی (مقدار برگشتی) تحویل داده شود.

`return` این تضمین را می‌دهد که آن آبمیوه (داده) تحویل داده شده و برنامه دقیقاً از همان جایی که منتظر مانده بود (خط `apple_juice = ...`)، ادامه پیدا کند.

خلاصه شیرین: `return` یعنی: «این نتیجه نهایی است. دستگاه خاموش. ادامه کار را دقیقاً از لحظه تحویل محصول از سر بگیرید».

۵. تنظیمات کارخانه (ورودی‌های پیش‌فرض)

گاهی اوقات ماجراجویان عجله دارند! آنها نمی‌خواهند هربار تمام جزئیات را به تابع بگویند. بلکه می‌خواهند تابع در حالت عادی کارش را انجام دهد، مگر اینکه دستور خاصی صادر شود.

به این قابلیت آرگومان پیش‌فرض^۵ می‌گوییم. درست مثل پیتزافروشی؛ اگر نگوید چه پنیری می‌خواهید، آشپز به صورت پیش‌فرض پنیر معمولی می‌ریزد. اما اگر تاکید کنید پنیر گودا، آن وقت دستور عوض می‌شود.

بیایید طلسم `print_divider` را ارتقا دهیم. معمولاً می‌خواهیم خط تیره = چاپ کند، اما اگر دلمان خواست، بتوانیم ستاره * یا \$ هم چاپ کنیم.

Commented [GG32]: علامت مساوی نیست مگه؟ چرا خط تیره؟

```
1. # 1. We set a default value inside the parenthesis: char=""
2. def print_divider(char=""):
3.     # This prints the character 20 times
4.     print(char * 20)
5.
6. # Scenario A: The Lazy Way (Using Default)
7. print("--- Standard Divider ---")
8. print_divider() # We gave NO argument, so Python uses "" automatically.
9.
10. # Scenario B: The Custom Way (Overriding Default)
11. print("--- Magic Divider ---")
12. print_divider("*") # We gave an argument, so "" is ignored and "*" is
    used.
13.
14. print("--- Rich Divider ---")
15. print_divider("$")
```

^۵Default Argument

خروجی:

```

--- Standard Divider ---
=====
--- Magic Divider ---
*****
--- Rich Divider ---
$$$$$$$$$$$$$$$$

```

تحلیل جادویی: در خط تعریف تابع: `def print_divider(char="")`، ما به پایتون گفتیم: «اگر کسی چیزی داخل پرانتز گذاشت، خودت `char` را مساوی `=` در نظر بگیر. اما اگر چیزی نوشتند، حرف آنها اولویت دارد». این ترفند باعث می‌شود توابع شما، هم ساده باشند (برای استفاده سریع) و هم قدرتمند (برای تنظیمات خاص).

نکته مخفی: جادوی تکثیر^۱: شاید در کد بالا متوجه یک طلسم عجیب شده باشید: `char * 20`. مگر می‌شود کلمات را ضرب کرد؟ در ریاضیات معمولی نه، اما در پایتون بله! علامت ستاره `*` وقتی بین دو عدد باشد، عمل ضرب را انجام می‌دهد (`2 * 3 = 6`). اما وقتی بین یک رشته و یک عدد قرار می‌گیرد، تبدیل به دستگاه فتوکپی می‌شود! این دستور، متن شما را به تعداد آن عدد تکرار می‌کند و به هم می‌چسباند.

مثال:

```

1. # 1. Repeating a laugh
2. print("Ha" * 3)
3. # Output: HaHaHa
4.
5. # 2. Making a long line (divider)
6. print("-" * 10)
7. # Output: -----

```

هشدار جادوگر:

یادتان باشد که رشته را فقط می‌توانید در عدد صحیح ضرب کنید.

- `"Ali" * 3` درست؛ علی‌علی‌علی؛
- `"Ali" * "Ali"` غلط؛ کلمه در کلمه ضرب نمی‌شود!
- `"Ali" * 2.5` غلط؛ نمی‌توانیم دو و نیم بار علی داشته باشیم!

^۱String Multiplication

۶. جادوی دقیق: برچسب زدن به مواد اولیه

گاهی وقت‌ها، طلسم‌های ما پیچیده می‌شوند و چند تا ورودی مختلف می‌گیرند. اگر ترتیب مواد اولیه را اشتباه بریزیم، ممکن است به جای اینکه **جون** بگیریم، سم بخوریم! برای جلوگیری از این فاجعه، پایتون به ما اجازه می‌دهد روی هر آرگومان یک برچسب بزنیم. یعنی موقع صدا زدن تابع، دقیقاً بگوییم کدام داده مال کدام پارامتر است. این طوری حتی اگر ترتیبشان را هم قاطی کنیم، پایتون گیج نمی‌شود.

به زبان کد:

```
1. # A spell to make a potion with specific ingredients
2. def brew_potion(herb, liquid):
3.     print("Mixing " + herb + " with " + liquid + "...")
4.
5. # Standard way (Order matters!):
6. brew_potion("Mint", "Water")
7.
8. # Keyword Arguments (Order doesn't matter, labels do!):
9. # We explicitly say which argument belongs to which parameter.
10. brew_potion(liquid="Dragon Blood", herb="Fire Flower")
```

Commented [GG33]: به نظرم بهتره جون باشه، تا جان

Commented [GG34]: کامنت ناشر: نیازی نیست به صورت کامنت خروجی‌هاشونو ببینیم؟

می‌بینید؟ با اینکه جایشان را عوض کردیم، چون برچسب (اسم پارامتر) داشتند، معجون درست ترکیب شد!

نکته مهم: به صورت ترکیبی هم می‌توانیم استفاده کنیم. این طوری که چند ورودی اول را به ترتیب بدهیم، ولی چند ورودی آخر را با اسمشان مشخص کنیم (برعکس این ترتیب شدنی نیست و ارور می‌دهد).

۷. پروژه: پاک‌سازی^۷ کد نفرین‌شده – 100 xp

ماجراجو، تو آماده‌ای! امروز مأموریت ما با همیشه فرق دارد. قرار نیست چیز جدیدی بسازیم، بلکه می‌خواهیم یک کد **بد** و تکراری که توسط یک جادوگر تنبل نوشته شده را با جادوی جدیدمان (توابع) پاک‌سازی و درمان کنیم.

۷-۱. تصویر نهایی: مشکل چیست و راه‌حل چه شکلی است؟

اول بیایید به کد نفرین‌شده^۷ صفحه بعد نگاه کنیم. هدف کد این است که مشخصات دو قهرمان را چاپ کند:

⁷ The Clean-Up Code

```

1. # --- The Bad, Repetitive Code ---
2. print("--- Adventurer ID Card ---")
3. print("Name: Aria")
4. print("Class: Ranger")
5. print("HP: 100")
6. print("=====")
7.
8. print("--- Adventurer ID Card ---")
9. print("Name: Kael")
10. print("Class: Mage")
11. print("HP: 80")
12. print("=====")

```

چرا این کد افتضاح است؟ ما ۵ خط کد را دقیقاً کپی/پیست کردیم! (فقط نام و مشخصات تغییر کرده است).

- اگر بخواهیم ۱۰۰ ماجراجو داشته باشیم، باید ۵۰۰ خط کد بنویسیم؟
 - اگر بخواهیم فرمت چاپ را عوض کنیم، باید ۱۰۰ جای مختلف را ویرایش کنیم؟ این یعنی فاجعه!
- راه حل:** می‌خواهیم یک ماشین چاپ کارت بسازیم.

تصور کن به جای نوشتن دستی، یک دستگاه داریم که فقط به آن می‌گوییم: «آریا، جادوگر، ۱۰۰» و دستگاه خودش تمام خطوط زیبا و کادربندی را چاپ می‌کند. این دستگاه، همان تابع است.

۷-۲. نقشه و طرح اولیه

بیایید منطق این دستگاه را مهندسی کنیم. هر تابع مثل یک کارخانه کوچک است:

۱. شناسایی الگوی تکراری (قالب): چه چیزی ثابت است؟

- خط‌کشی‌ها --- و ===؛

- کلمات Name:، Class: و HP:.

۲. شناسایی تغییرات (ورودی‌ها): چه چیزی در هر کارت عوض می‌شود؟

- اسم (name):
- رتبه (job_class):
- جان (hp).

اینها پارامترهای تابع ما هستند (مواد اولیه‌ای که داخل دستگاه می‌ریزیم).

۳. ساخت دستگاه (def): یک بار برای همیشه دستورات چاپ را داخل یک بسته به نام `show_adventurer_stats` می‌نویسیم.

۴. استفاده از دستگاه (Call): در برنامه اصلی، فقط نام تابع را صدا می‌زنیم و مواد اولیه را به آن می‌دهیم.

۷-۳. چالش‌های فکری

قبل از دیدن کد تمیز، سعی کن این معماها را حل کنی:

- چالش ۱. تعریف تابع: چطور به پایتون می‌فهمانی که داری یک ابزار می‌سازی و نه یک کد معمولی؟ با کدام کلمه کلیدی؟ (راهنمایی: `def`).
- چالش ۲. جعبه‌های ورودی: تابع ما به ۳ تکه اطلاعات نیاز دارد تا کار کند. این ۳ متغیر را داخل پرانتز تعریف تابع () با چه اسم‌هایی می‌گذاری؟ (مثال: `(name, job, hp)`).
- چالش ۳. چسباندن متن و عدد (تله خطرناک!): در خط چاپ HP، یک متن ثابت ("HP: ") داریم و یک عدد hp که مثلاً ۱۰۰ است. اگر بنویسی `hp + "HP: "` پایتون ارور می‌دهد (چون نمی‌تواند عدد را به متن بچسباند). راه حل چیست؟ (راهنمایی: استفاده از کیمیاگری `str(hp)` برای تبدیل عدد به متن).

۷-۴. کد نهایی پروژه (The Clean Spell)

وقت پاک‌سازی است! کد زیر را در فایل `project.py` بنویس. ببین چقدر برنامه اصلی (پایین کد) کوتاه و تمیز شده است.

```
1. # --- The Cleaned-Up Spellbook (Using Functions) ---
2.
3. # 1. --- Define the Reusable Spell (The Machine) ---
4. # We create ONE function that takes 3 arguments (inputs)
5. def show_adventurer_stats(name, job_class, hp):
6.     print("--- Adventurer ID Card ---")
7.     print("Name: " + name)
8.     print("Class: " + job_class)
9.
10.    # CRITICAL: We must convert integer 'hp' to string 'str()' to join
    it!
11.    print("HP: " + str(hp))
12.
13.    print("=====")
14.
15.
16. # 2. --- Call the Spell (Using the Machine) ---
17. # Our main code is now simple, clean, and easy to read!
18.
```

```
19. # Creating Aria's card
20. show_adventurer_stats("Aria", "Ranger", 100)
21.
22. # Creating Kael's card
23. show_adventurer_stats("Kael", "Mage", 80)
24.
25. # Want to add a new one? It's just ONE line now!
26. show_adventurer_stats("Gimli", "Warrior", 150)
```

اجرا کن و لذت ببر! خروجی این کد تمیز، دقیقاً مشابه آن کد کثیف و تکراری است. اما حالا تو یک معماری. اگر بخواهی طرح کارت‌ها را عوض کنی، فقط کافی‌ست یک بار (داخل تابع) تغییر ایجاد کنی تا کارت تمام ۱۰۰۰ نفر ماجراجو تغییر کند. این یعنی قدرت!

۷-۵. جشن پیروزی و تحلیل

آفرین! همین الان از یک کارآموز که کدها را پشت‌سرهم می‌نویسد، به یک برنامه‌نویس ساختاریافته تبدیل شدی. تو یاد گرفتی:

- چطور جلوی تکرار را بگیری. قانون DRY: Don't Repeat Yourself؛
 - چطور کدهای طولانی را در بسته‌های کوچک و قابل‌مدیریت `def` دسته‌بندی کنی.
- ماجرایابی‌های بزرگ با همین ابزارهای کوچک ساخته می‌شوند. کوله‌پشتی‌ات را بردار، فصل بعدی منتظر است!

| 🌸 ارتقاء لول: 🌸 |

۲۲ → ۳۰

مهارت‌های جدید:

◆ دستور Def

له نام باستانی: خلق طلسم شخصی.
توضیح: کارهایی که تکرار می‌شدند، اکنون در یک کلمه خلاصه شده‌اند.

◆ دستور Return

له نام باستانی: بازگشت کیوتر نامه‌بر.
توضیح: تابعی که دست خالی بر نمی‌گردد و نتیجه را تقدیم می‌کند.

فصل ۹: طلسم محافظتی: ساخت رمز عبور امن

مأموریت فصل: تولیدکننده رمز برای یک صندوقچه گنج.

نقشه راه:

- استفاده از قدرت‌های مخفی پایتون: کتابخانه‌ها (Modules) و طلسم `import`;
- کتابخانه `random`: بهترین دوست ما برای کارهای تصادفی;
- جادوی شمارش: تکرار یک طلسم با `for i in range()`;
- پروژه: ساخت برنامه‌ای برای تولید رمزهای عبور قوی و تصادفی.

Commented [GG35]: فونت کد؟

۱. قدرت‌های مخفی پایتون: کتابخانه‌ها و Import

تا الان هر کدی نوشتیم، دسترنج خودمان بود که در فایل *project.py* می‌نوشتیم. اما پایتون موقع نصب، هزاران خط کد آماده و حرفه‌ای را نیز همراه خودش روی کامپیوتر شما نصب کرده‌است. به این مجموعه عظیم، کتابخانه استاندارد^۱ می‌گوییم.

این کدها در بسته‌هایی جداگانه به نام ماژول^۲ دسته‌بندی شده‌اند.

تصورکن: فایل *project.py* مثل میز کار شماست و ماژول‌ها مانند جعبه ابزارهای تخصصی در انبارند (مثلاً جعبه ابزار ریاضی، جعبه ابزار شانس و...). شما همه ابزارها را همیشه روی میز نمی‌ریزید! هر وقت به هر کدام نیاز داشته باشید، با دستور `import` آن را از انبار به روی میز می‌آورید.

```
1. # Go to the library and bring the 'random' toolbox
2. import random
```

با این خط ساده، به پایتون می‌گوییم: «جعبه ابزار random را بیاور، من می‌خواهم از ابزارهای داخلش استفاده کنم».

۲. جعبه ابزار random: بهترین دوست ما برای کارهای تصادفی

حالا که جعبه ابزار روی میز است، چطور ابزار خاصی را از داخلش برداریم؟ ما از نقطه (.) استفاده می‌کنیم. این نقطه یعنی دسترسی به محتویات جعبه. فرمول آن این است:

```
ToolboxName.ToolName
```

در اینجا ۳ ابزار بسیار پرکاربرد از جعبه random را بررسی می‌کنیم:

۱. ابزار `random.randint(a, b)`: تاس دیجیتال

این ابزار یک عدد صحیح تصادفی بین دو عددی که به او می‌دهی انتخاب می‌کند.

نکته مهم: در پایتون، این ابزار برخلاف بقیه، شامل خود عدد آخری هم می‌شود. دقیقاً مثل تاس!

```
1. import random
2.
3. # Roll a simple 6-sided die
4. # This will pick a number: 1, 2, 3, 4, 5, or 6
5. die_roll = random.randint(1, 6)
6.
```

^۱ Standard Library

^۲ Module

```
7. print("You rolled a:")
8. print(die_roll)
```

۲. ابزار random.choice(sequence): انتخاب شانسی

این محبوب‌ترین ابزار ما در بازی‌سازی است. یک لیست به او می‌دهی و او چشمانش را می‌بندد و یک آیتم را شانسی بیرون می‌کشد.

```
1. import random
2.
3. # A chest of possible loot
4. loot_options = ["Gold Coin", "Health Potion", "Rusty Dagger", "Diamond"]
5.
6. # Use the 'choice' tool to pick ONE item blindly
7. found_item = random.choice(loot_options)
8.
9. print("You opened the chest and found a...")
10. print(found_item)
11. # Try running this multiple times!
```

۳. ابزار random.shuffle(list): بُرزدن کارت‌ها

این ابزار یک لیست را می‌گیرد و آیتم‌های آن را درجا بُر می‌زند (ترتیبشان را به هم می‌ریزد).

هشدار: این ابزار خروجی جدیدی نمی‌سازد (Return نمی‌کند)، بلکه خود لیست اصلی را تغییر می‌دهد. پس ترتیب قبلی برای همیشه از بین می‌رود!

```
1. import random
2.
3. # A list of party members
4. party = ["Aria", "Kael", "Lyra", "Zane"]
5.
6. print("Original order:")
7. print(party)
8.
9. # Shuffle the list in-place (Changes the 'party' list directly)
10. random.shuffle(party)
11.
12. print("New battle order:")
13. print(party)
```

اجرا کن و نتیجه را ببین! هربار که کد shuffle را اجرا کنی، اعضای گروه در ترتیب جدیدی قرار می‌گیرند.

۳. جادوی شمارش: تکرار با `for i in range()`

Commented [GG36]: بهتره فونت تیتراشه یا کد؟

ما یاد گرفتیم که چطور با حلقه `for` تمام جیب‌های کوله‌پشتی (`List`) را بگردیم (مثلاً `for item in backpack`). اما اگر لیستی در کار نباشد چه؟ اگر فقط بخواهیم یک کار خاص را دقیقاً ۱۰ بار تکرار کنیم (مثلاً ۱۰ باره در ضربه بزنیم)، باید چه کار کرد؟

برای این کار، ما از یک ابزار داخلی قدرتمند به نام `range()` استفاده می‌کنیم. دستور `range(5)` یک دنباله نامرئی از اعداد می‌سازد.

نکته مهم: در دنیای کامپیوتر، شمارش همیشه از صفر شروع می‌شود. پس `range(5)` اعداد ۱ تا ۵ را نمی‌سازد، بلکه ۰ تا ۴ را می‌سازد (که در مجموع می‌شود ۵ عدد).

Commented [GG37]: عددهای این پاراگراف فارسی باشن یا انگلیسی و با فونت کد؟

بیایید امتحان کنیم:

```
1. # The range(stop) tool
2. # We want to repeat an action 5 times
3. # This generates numbers: 0, 1, 2, 3, 4
4.
5. for i in range(5):
6.     print("This is loop number: " + str(i))
```

متغیر `i` چیست؟

در کد بالا، `i` (که مخفف Index است) مثل یک شمارنده در دست داور مسابقه است.

- در دور اول، مقدارش ۰ است.
- در دور دوم، مقدارش ۱ است.
- و همین‌طور ادامه می‌یابد تا به ۴ برسد و حلقه تمام شود.

نکته: اسم این متغیر لازم نیست حتماً `i` باشد. شما می‌توانید بنویسید `for number in range(5)` یا هر اسم دیگری، اما استفاده از `i` در بین برنامه‌نویسان یک رسم قدیمی است.

۴. پروژه: ساخت قفل رمزنگاری شده^۳ - 100 xp

ماجراجو، تو آماده‌ای!

^۳ Password Generator

بالاخره یک صندوقچه گنج پیدا کردیم و می‌خواهیم غنائم خود را در آن پنهان کنیم. اما یک مشکل بزرگ داریم: دزدهای امروزی باهوش‌اند و رمزهای ساده مثل 1234 یا password را سریع حدس می‌زنند. ما به یک طلسم نیاز داریم که یک رمز عبور ۱۲ حرفی، قوی و کاملاً تصادفی برایمان بسازد. رمزی که حتی خودمان هم نتوانیم حدس بزنیم!

۴-۱. تصویر نهایی: این ماشین رمزساز چطور کار می‌کند؟

قبل از کدنویسی، تصور کن یک دستگاه جادویی داری که شبیه گردونه شانس است.

۱. دکمه را فشار می‌دهی؛

۲. دستگاه دستش را توی یک کیسه بزرگ پر از حروف و اعداد می‌کند.

و ۱۲ بار، هربار یک مهره را شانسی بیرون می‌کشد و کنار هم می‌چیند.

نتیجه چیزی مثل !Kj9#mP2\$aL5 می‌شود. این رمز غیرقابل نفوذ است، چون هیچ الگوی خاصی ندارد.

۴-۲. نقشه و طرح اولیه

بیایید مواد لازم برای این طلسم را بررسی کنیم. ما می‌خواهیم یک **معجون** بسازیم:

- **وارد کردن جادوی خارجی (Import):** پایتون خودش بلد نیست تاس بیندازد. ما باید یک کتابخانه کمکی به نام **random** را احضار کنیم.
- **آماده‌سازی مواد اولیه^۴:** یک رمز قوی باید مخلوطی از ۴ چیز باشد:
 - حروف کوچک (a-z)؛
 - حروف بزرگ (A-Z)؛
 - اعداد (0-9)؛
 - نمادهای عجیب (!@#\$).
- **ترکیب مواد:** ما همه این ۴ گروه را با عملگر جمع + به هم می‌چسبانیم و داخل یک ظرف بزرگ (متغیر `all_characters`) می‌ریزیم.

^۴The Ingredients

- **حلقه تکرار و انتخاب:** حالا ۱۲ بار این کار را تکرار می‌کنیم:
 - دست در ظرف کن، یک کاراکتر شانس بردار (choice) و به رمز نهایی اضافه کن.

[GG38] Commented: فونت کد درسته؟

۳-۴. چالش‌های فکری

قبل از دیدن کد، سعی کن این گره‌های ذهنی را باز کنی:

- **چالش ۱. احضار کتابخانه:** در خط اول برنامه، با چه دستوری به پایتون می‌گویی که می‌خواهی از ابزارهای تصادفی استفاده کنی؟ (راهنمایی: `import`).
- **چالش ۲. چسباندن رشته‌ها:** ما ۴ متغیر متنی داریم. چطور همه را به هم وصل کنیم و در یک متغیر جدید بریزیم؟ (راهنمایی: در دنیای رشته‌ها، علامت `+` یعنی چسباندن).
- **چالش ۳. حلقه شمارشی:** ما دقیقاً ۱۲ بار تکرار می‌خواهیم. کدام نوع حلقه برای زمانی که تعداد دقیق را می‌دانیم مناسب‌تر است؟ `while` یا `for`؟ (راهنمایی: `for` همراه با `range`).
- **چالش ۴. انتخاب شانس:** وقتی مواد را مخلوط کردیم، با کدام دستور از کتابخانه `random` می‌توانیم فقط یک دانه از حروف را به صورت شانس بیرون بکشیم؟ (راهنمایی: `random.choice`).

[GG39] Commented: علامت یا عملگر؟

۴-۴. کد نهایی پروژه (The Spell)

وقت ساختن قفل است. کد زیر را در فایل `project.py` بنویس. به نحوه استفاده از `random` و ساختن تدریجی رمز عبور دقت کن.

[GG40] Commented: فونت؟

```
1. import random # We need this wizard to generate random things!
2.
3. # --- 1. The Ingredients (Manual Entry) ---
4. # We define the 4 types of characters we want in our password
5. lowercase_letters = "abcdefghijklmnopqrstuvwxyz"
6. uppercase_letters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
7. numbers = "0123456789"
8. symbols = "!@#$%^&*()_+-=[]{}|;:,.<.>?"
9.
10. # --- 2. Mixing the Soup ---
11. # Combine all ingredients into one big pool of characters
12. all_characters = lowercase_letters + uppercase_letters + numbers + symbols
13.
14. # --- 3. Magic Configuration ---
15. password_length = 12 # How long should the password be?
```



```

16. password = ""          # We start with an empty string
17.
18. # --- 4. Casting the Spell (The Loop) ---
19. # We loop 12 times. inside the loop, we pick one char and add it.
20. for _ in range(password_length):
21.     # Pick one random character from the big pool
22.     random_char = random.choice(all_characters)
23.
24.     # Add it to our password variable
25.     password = password + random_char # OR: password += random_char
26.
27. # --- 5. Reveal the Secret ---
28. print("-----")
29. print(f"Your secure password is: {password}")
30. print("-----")

```

Commented [GG41]: کامنت ناشر: درسته؟

اجرا کن و نتیجه را ببین! برنامه را چند بار پشت سر هم اجرا کن.

می‌بینی؟ هر بار یک رمز کاملاً متفاوت و عجیب ساخته می‌شود. حالا می‌توانی این رمز را برای صندوقچه گنج (یا ایمیل واقعی خودت!) استفاده کنی. (البته یادت باشه اگر رمزی را برایت ساخت و آن را فراموش کردی، دیگر نمی‌توانی برنامه را اجرا کنی و دوباره همان رمز را دریافت کنی).

۴-۶. جشن پیروزی و تحلیل

تبریک! تو الان امنیت را تضمین کردی.

در این پروژه یاد گرفتی:

۱. چطور از کتابخانه‌های خارجی استفاده کنی (import)؛

۲. چطور با رشته‌ها مثل مهره‌های بازی رفتار کنی (بجسبانی و انتخاب کنی)؛

۳. و چطور شانس را وارد برنامه‌نویسی کنی.

حالا هیچ دزدی نمی‌تواند الگوی رمزهای تو را پیدا کند! ماجراجویی ادامه دارد...

| 🌸 ارتقاء لول: 🌸 |
۳۰ → ۳۵

مهارت‌های جدید:

◆ دستور Import

له نام باستانی: دروازه ابعاد.

توضیح: اتصال به کتابخانه جهانی پایتون و استفاده از قدرت دیگران.

◆ کتابخانه Random

له نام باستانی: تاس سرنوشت.

توضیح: افزودن عنصر شانس و غیرقابل پیش بینی بودن به جهان منظم.

◆ سطح جدید: E

توضیح: تبریک! به سطح جدیدی رسید.

۵. مأموریت‌های جانبی: تالار تمرین

ماجراجوی عزیز! قبل از اینکه وارد سرزمین‌های ناشناخته بخش سوم شوی، باید در تالار تمرین مهارت‌هایت را در کار با لیست‌ها، دیکشنری‌ها و توابع تقویت کنی. اینجا ۱۰ چالش پشت‌سرهم منتظر توست.

قانون تالار ساده است؛ تا وقتی کد را خودت ننوشتی، به پاسخ‌نامه نگاه نکن!

۵-۱. مأموریت ها

۱. مأموریت: شبیه ساز تاس^۶ - 20 xp

در بازی های رومیزی، همیشه به تاس نیاز داریم.

- **وظیفه:** تابعی بنویس که وقتی اجرا می شود، یک عدد تصادفی بین ۱ تا ۶ را برگرداند و چاپ کند.
- **هدف:** تمرین کار با ماژول random و تعریف تابع ساده.

۲. مأموریت: جعبه شانسی^۷ - 20 xp

پادشاه های بازی معمولاً تصادفی اند.

- **وظیفه:** یک لیست از آیتم ها (مثل "Gold"، "Sword"، "Potion" و "Shield") بساز. برنامه باید به صورت تصادفی یکی از آنها را انتخاب کرده و به عنوان جایزه به کاربر بدهد.
- **هدف:** استفاده از دستور choice از ماژول random.

۳. مأموریت: شمارشگر حروف صدادار^۸ - 20 xp

تحلیلگران متن نیاز دارند بدانند یک جمله چقدر خوش آهنگ است.

- **وظیفه:** یک کلمه یا جمله از کاربر بگیر. برنامه باید بشمارد که چند حرف صدادار (a, e, i, o, u) در آن وجود دارد.
- **هدف:** استفاده از حلقه for روی رشته ها و شرط ها.

۴. مأموریت: آینه یاب جادویی (Palindrome Checker) - 20 xp

برخی کلمات جادویی اند و از هر دو طرف یکسان خوانده می شوند (مثل "radar" یا "level").

- **وظیفه:** تابعی بنویس که یک کلمه را بگیرد. اگر کلمه با برعکس خودش برابر بود، بگوید: «(Magic Word!)»، وگرنه بگوید: «(Normal Word)».

Commented [GG42]: نام انگلیسی اکثر مأموریت ها دقیقاً ترجمه فارسی عنوان آنها نیست. میشه که همه شون به جای پانویس در خود متن و جلوی اسم فارسی بیان. نظرتون؟

^۶ Dice Roller

^۷ Loot Box Generator

^۸ Vowel Counter

- هدف: کار با اسلایسینگ^۹ رشته‌ها [1 - :] و معکوس کردن آنها.

۵. مأموریت: کلاه گروه‌بندی^{۱۰} - 20 xp

به مدرسه جادوگری هاگوارتز خوش آمدی!

- وظیفه: یک لیست از گروه‌ها (Slytherin و Ravenclaw, Hufflepuff, Gryffindor) داشته باش. اسم دانش‌آموز را بگیر و به صورت تصادفی او را در یکی از این گروه‌ها قرار بده.

- هدف: ترکیب گرفتن ورودی و انتخاب تصادفی.

۶. مأموریت: حذف تکراری‌ها^{۱۱} - 20 xp

کوله‌پشتی ما پر از آیتم‌های تکراری شده‌است.

- وظیفه: یک لیست با اعداد تکراری بساز (مثلاً [1, 2, 2, 3, 4, 4]). برنامه‌ای بنویس که تکراری‌ها را حذف کند و یک لیست تمیز (مثلاً [1, 2, 3, 4]) تحویل دهد.

- هدف: ساختن یک لیست جدید و بررسی وجود آیتم‌ها با شرط `if ... not in`

۷. مأموریت: تبدیل گرایموجی^{۱۲} - 20 xp

پیام‌ها خشک و بی‌روح‌اند، بیایید به آنها احساس بدهیم.

- وظیفه: یک دیکشنری بساز که کلمات را به نماد تبدیل کند (مثلاً "happy" به ":) " و "sad" به "(: "). جمله‌ای از کاربر بگیر و کلمات کلیدی را با نمادها جایگزین کن.
- هدف: کار با متد `split` در رشته‌ها و جست‌وجو در دیکشنری.

۸. مأموریت: سنگ، کاغذ، قیچی^{۱۳} - 20 xp

بازی کلاسیک دونفره، اما این بار حریف تو کامپیوتر است.

^۹ Slicing

^{۱۰} The Sorting Hat

^{۱۱} Duplicate Remover

^{۱۲} Emoji Converter

^{۱۳} Rock, Paper, Scissors

- **وظیفه:** انتخاب کاربر (سنگ/کاغذ/قیچی) را بگیر. کامپیوتر هم به صورت اتفاقی^{۱۴} انتخاب می‌کند. سپس طبق قوانین بازی، برنده را مشخص کن.
- **هدف:** مدیریت شرط‌های تودرتو و منطق بازی.

۹. مأموریت: فاکتور خرید^{۱۵} – 20 xp

صندوقدار فروشگاه میوه نیاز به کمک دارد.

- **وظیفه:** یک دیکشنری از قیمت‌ها داشته باش (مثلاً `Banana: 5, Apple: 10`). یک لیست خرید هم داری (مثلاً `["Apple", "Apple", "Banana"]`). برنامه باید مجموع قیمت این لیست خرید را حساب کند.
- **هدف:** ترکیب پیمایش لیست (`for`) و خواندن مقدار از دیکشنری.

۱۰. مأموریت: خالق شخصیت^{۱۶} – 20 xp

برای بازی نقش‌آفرینی جدیدمان نیاز به تولید انبوه دشمن و قهرمان داریم.

- **وظیفه:** تابعی بنویس که هیچ ورودی‌ای نگیرد، اما در خروجی، یک دیکشنری برگرداند که شامل `Name, Role, Health` و `Power` باشد (مقادیر باید از لیست‌های ازپیش‌آماده به صورت اتفاقی انتخاب شوند).
- **هدف:** ساخت ساختمان داده‌های پیچیده (دیکشنری) درون توابع.

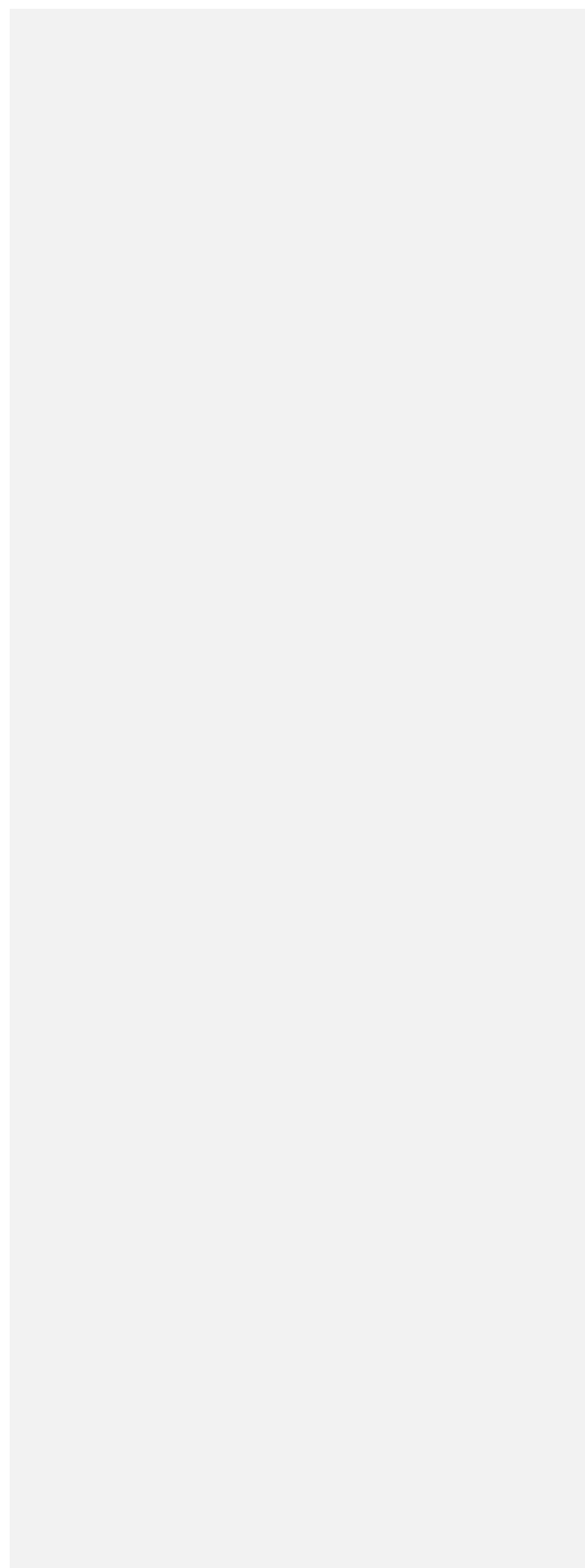
آیا سفرت را در این مرحله کامل کرده‌ای، ماجراجو؟

اگر بله، کتاب را ورق بزن؛ کتابخانه سایه‌ها (نتیجه مأموریت‌ها) در صفحه بعد انتظار تو را می‌کشد.

¹⁴ Randomly

¹⁵ Shopping Cart

¹⁶ Character Generator



۵-۲. کتابخانه سایه‌ها (نتیجهٔ مأموریت‌ها)

۱. مأموریت: شبیه‌ساز تاس

نکته: برای تولید عدد صحیح تصادفی از **randint** استفاده می‌کنیم.

```
1. import random
2.
3. def roll_dice():
4.     number = random.randint(1, 6)
5.     print(f"You rolled a: {number}")
6.
7. # Call the function
8. roll_dice()
```

۲. مأموریت: جعبهٔ شانسی

نکته: متد **choice** بهترین ابزار برای انتخاب یک آیتم از لیست است.

```
1. import random
2.
3. items = ["Gold", "Sword", "Potion", "Shield"]
4. reward = random.choice(items)
5.
6. print(f"You opened the box and found: {reward}")
```

۳. مأموریت: شمارشگر حروف صدادار

نکته: می‌توانیم چک کنیم آیا حرف در رشتهٔ "aeiou" وجود دارد یا خیر.

```
1. text = input("Enter a sentence: ")
2. vowels = "aeiou"
3. count = 0
4.
5. for char in text:
6.     if char.lower() in vowels:
7.         count += 1
8.
9. print(f"Number of vowels: {count}")
```

۴. مأموریت: آینه‌یاب جادویی

نکته: در پایتون [:-1] کل رشته را برعکس می‌کند.

```
1. def check_palindrome(word):
2.     # Check if word equals its reverse
3.     if word == word[::-1]:
4.         print("Magic Word! (It is a palindrome)")
5.     else:
6.         print("Normal Word.")
7.
8. user_word = input("Enter a word: ")
```


9. check_palindrome(user_word)

Commented [GG43]: بهتره به صفحه قبل بره

۵. مأموریت: کلاه گروه‌بندی

نکته: با لیست‌ها می‌توانیم هر تعداد گروه که بخواهیم تعریف کنیم.

```
1. import random
2.
3. houses = ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]
4. name = input("Enter student name: ")
5.
6. assigned_house = random.choice(houses)
7. print(f"{name} belongs to... {assigned_house}!")
```

۶. مأموریت: حذف تکراری‌ها

نکته: ما یک لیست خالی می‌سازیم و فقط اعدادی که قبلاً ندیده‌ایم را به آن اضافه می‌کنیم.

```
1. numbers = [1, 2, 2, 3, 4, 4, 5]
2. unique_numbers = []
3.
4. for num in numbers:
5.     if num not in unique_numbers:
6.         unique_numbers.append(num)
7.
8. print(f"Clean list: {unique_numbers}")
```

۷. مأموریت: تبدیل گرایموجی

نکته: متد get در دیکشنری باعث می‌شود اگر کلمه‌ای پیدا نشد، بتوانیم خود کلمه را بدون تغییر برگردانیم (جلوگیری از خطا).

```
1. message = input("Type your message: ")
2. words = message.split(" ")
3.
4. emojis = {
5.     "happy": ":)",
6.     "sad": ":("
7. }
8.
9. output = ""
10. for word in words:
11.     output += emojis.get(word, word) + " "
12.
13. print(output)
```

۸. مأموریت: سنگ، کاغذ، قیچی

نکته: منطق بازی را با شرط‌های if/elif پیاده‌سازی می‌کنیم.

```

1. import random
2.
3. options = ["rock", "paper", "scissors"]
4. computer = random.choice(options)
5. user = input("Choose (rock, paper, scissors): ")
6.
7. print(f"Computer chose: {computer}")
8.
9. if user == computer:
10.     print("It's a tie!")
11. elif user == "rock" and computer == "scissors":
12.     print("You win!")
13. elif user == "paper" and computer == "rock":
14.     print("You win!")
15. elif user == "scissors" and computer == "paper":
16.     print("You win!")
17. else:
18.     print("You lose!")

```

۹. مأموریت: فاکتور خرید

نکته: اینجا از لیست برای اقلام و از دیکشنری برای قیمت‌ها استفاده کرده‌ایم.

```

1. prices = {"Apple": 10, "Banana": 5, "Orange": 7}
2. cart = ["Apple", "Apple", "Banana"]
3.
4. total = 0
5. for item in cart:
6.     if item in prices:
7.         total += prices[item]
8.
9. print(f"Total Bill: ${total}")

```

۱۰. مأموریت: خالق شخصیت

نکته: دیکشنری خروجی می‌تواند مقادیر خود را از توابع دیگری انتخاب‌های تصادفی بگیرد.

```

1. import random
2.
3. def create_character():
4.     names = ["Arthur", "Merlin", "Lancelot"]
5.     roles = ["Warrior", "Mage", "Archer"]
6.
7.     character = {
8.         "Name": random.choice(names),
9.         "Role": random.choice(roles),
10.        "Health": random.randint(50, 100),
11.        "Power": random.randint(10, 20)
12.    }
13.    return character
14.
15. # Create a new hero
16. hero = create_character()
17. print(hero)

```